AI-POWERED SMART SERVICE MARKETPLACE FOR HOUSEHOLD MAINTENANCE IN SRI LANKA

INDIVIDUAL THESIS

Wijebandara W.N.S.K

(IT22643704)

Individual component: Automated Review Credibility and Sentiment Analysis

Group ID: R26-IT-090

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Wijebandara W.N.S.K

Student ID: IT22643704

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Dasuni Navinna		10/10/2026
Co-supervisor: Mr. Nelum Chathuranga Amarasena		10/10/2026

ABSTRACT

Online marketplaces for household services rely on star ratings, but star ratings alone are a weak signal: they are skewed towards high values, they often disagree with what customers write, and an average based on a few reviews is unreliable. In Sri Lanka, customers also write reviews in English, Sinhala script and Singlish (romanised, code-mixed Sinhala), which English-only tools cannot interpret. This thesis presents Component 3 of an AI-powered household-service marketplace: an automated review credibility and sentiment analysis service that produces a category-specific trust score for each worker. Reviews are accepted only after a job is completed, which prevents fake reviews. A three-class (Negative, Neutral, Positive) dataset of 101,217 rows was built by extending a public English Twitter dataset with Sinhala machine translations, rule-based Sinhala-to-Singlish transliterations and template-generated Singlish sentences. The multilingual transformer xlm-roberta-base was fine-tuned on 67,411 rows and reached 95.41% accuracy, a weighted F1 score of 0.954 and per-class F1 scores between 0.951 and 0.958 on 16,853 held-out rows. It classified positive and negative sentences correctly in all three language forms, but misclassified two of three neutral test sentences and showed overconfidence after the fifth epoch. Each review's sentiment is converted into a confidence-weighted value between -1 and +1 and fused with the star ratings (60% stars, 40% text). A Bayesian credibility factor, N/(N+5), pulls scores based on few reviews towards a neutral prior of 6.0 out of 10, and scores are kept separate for each service category. The model and the scoring logic run as a FastAPI service that supplies the trust score to the worker recommendation engine. Because the training data is augmented and comes from tweets rather than service reviews, the reported accuracy is likely to be optimistic; validation on real Singlish service reviews is the main direction for future work.

Keywords: Sentiment analysis, Singlish, XLM-RoBERTa, Bayesian average, Trust score

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Prof. Dasuni Navinna, and my co-supervisor, Mr. Nelum Chathuranga Amarasena, for their guidance, encouragement and constructive feedback throughout this research. Their advice shaped both the direction of this component and the way it is presented in this thesis.

I am grateful to the members of research group R26-IT-090 for their collaboration in designing and building the shared marketplace application, and for the many discussions that helped define how this component works with the rest of the system.

I also thank the lecturers and staff of the Department of Information Technology, Sri Lanka Institute of Information Technology, for the knowledge and support provided during my degree, and the creators of the public dataset and open-source tools on which this work is built.

Finally, I thank my family and friends for their patience, support and encouragement throughout this project.

TABLE OF CONTENTS

1 INTRODUCTION *1*

1.1 Overall project context **1**

1.2 Individual component context **2**

1.3 Problem statement **3**

1.4 Research gap **4**

1.5 Aim and objectives **5**

1.6 Research questions or hypotheses **5**

1.7 Scope limitations and assumptions **6**

1.8 Individual contribution **7**

1.9 Thesis structure **7**

2 LITERATURE REVIEW AND RELATED WORK *9*

2.1 Review method and scope **9**

2.2 Domain and theoretical foundations **9**

2.2.1 Sentiment analysis 9

2.2.2 Transformers and multilingual language models 10

2.2.3 Evaluation of classifiers 11

2.2.4 Ratings, reputation and Bayesian averaging 11

2.3 Related research **12**

2.3.1 Sinhala and code-mixed sentiment analysis 12

2.3.2 Data augmentation for low-resource text 13

2.3.3 Disagreement between review text and star rating 13

2.3.4 Fake reviews and opinion spam 14

2.4 Existing systems products and techniques **14**

2.5 Gap synthesis and conceptual framework **15**

3 RESEARCH METHODOLOGY *17*

3.1 Research design **17**

3.2 Data participants or study materials **18**

3.2.1 Base dataset 18
3.2.2 Augmentation with Sinhala and Singlish text 19
3.2.3 Data cleaning and label encoding 20
3.2.4 Train and test split 21
3.3 Experimental or study procedure 21
3.3.1 Training environment 21
3.3.2 Model and tokenisation 22
3.3.3 Training settings 22
3.3.4 Procedure 22
3.4 Measures and evaluation metrics 23
3.5 Data analysis 24
3.6 Ethics privacy security and responsible AI 24
3.7 Reproducibility and research artefacts 25
4 PRODUCT AND SOLUTION DESIGN 26
4.1 Requirements 26
4.2 Component architecture within the complete solution 27
4.3 Design alternatives and decisions 28
4.4 Detailed design 30
4.4.1 Sentiment value of a review 30
4.4.2 Star score and sentiment score 30
4.4.3 Fusion 30
4.4.4 Credibility and Bayesian adjustment 31
4.4.5 Worked example 32
4.4.6 Category isolation 32
4.4.7 Data design 32
4.4.8 Service interface 32
4.5 Standards and professional practice 33
5 IMPLEMENTATION INTEGRATION AND TESTING 34
5.1 Development environment and technology stack 34
5.2 Component implementation 34
5.2.1 Data augmentation scripts 34

5.2.2 Training notebook 35
5.2.3 Sentiment prediction 36
5.2.4 Trust-score calculation 37
5.3 Integration with the other components 37
5.4 Verification and software testing 38
5.5 Deployment and operation 39
6 RESULTS AND VALIDATION 41
6.1 Research results 41
6.1.1 Training progress 41
6.1.2 Overall and per-class performance 42
6.1.3 Confusion matrix 42
6.1.4 Behavioural smoke tests 43
6.1.5 Answer to RQ1 44
6.2 Product and technical results 45
6.3 User or stakeholder validation 46
6.4 Integration results 46
6.5 Summary of findings 47
7 DISCUSSION 48
7.1 Interpretation of findings 48
7.2 Comparison with literature baselines and existing solutions 49
7.3 Research contribution 50
7.4 Product contribution 50
7.5 Limitations and threats to validity 51
7.6 Implications 52
8 COMMERCIALISATION AND SUSTAINABILITY 53
8.1 Value proposition and target users 53
8.2 Market and competitive position 53
8.3 Business or sustainability model 54

8.4 Scalability intellectual property and compliance **55**

8.5 Contribution to the Sustainable Development Goals **56**

9 CONCLUSION AND FUTURE DIRECTIONS ***57***

9.1 Conclusion **57**

9.2 Achievement of objectives **57**

9.3 Individual research and product contributions **58**

9.4 Future directions **58**

REFERENCES ***60***

GLOSSARY ***65***

APPENDICES ***66***

Appendix A Extended results and research artefacts **66**

A.1 Training run summary 66

A.2 Template Singlish generator settings 66

A.3 Transliteration examples 66

Appendix B Academic integrity and AI use disclosure **68**

LIST OF FIGURES

Figure 1.1: Overall architecture of the smart service marketplace and the position of Component 3 2

Figure 2.1: Conceptual framework of the review credibility and sentiment component 16

Figure 3.1: Data preparation and model training pipeline 18

Figure 3.2: Label distribution of the augmented dataset 20

Figure 4.1: Review processing flow in Component 3 28

Figure 4.2: Credibility factor and its effect on the final trust score 31

Figure 6.1: Training and validation loss, accuracy and weighted F1 per epoch 41

Figure 6.2: Confusion matrix of the final model on the test set 43

Figure 6.3: Raw and final trust scores for the eight scenarios 45

LIST OF TABLES

Table 2.1: Rating approaches of existing platforms and techniques 14

Table 3.1: Label distribution of the augmented dataset 20

Table 3.2: Training and test sets 21

Table 3.3: Training hyperparameters 22

Table 4.1: Functional requirements of Component 3 26

Table 4.2: Non-functional requirements of Component 3 26

Table 4.3: Design alternatives and decisions 29

Table 5.1: Technology stack 34

Table 5.2: Test plan and status 38

Table 5.3: Scoring formula verification cases 39

Table 6.1: Per-epoch training results 41

Table 6.2: Per-class results of the final model on the test set 42

Table 6.3: Confusion matrix (rows are true labels, columns are predicted labels) 43

Table 6.4: Smoke-test and interactive-test predictions 43

Table 6.5: Trust-score scenarios 45

Table 7.1: Comparison with related sentiment studies 49

Table 8.1: Competitive comparison of trust features 54

Table 9.1: Achievement of objectives 58

Table A.1: Summary of the final training run 66

Table A.2: Settings of the template Singlish generator 66

Table A.3: Examples of the rule-based Sinhala-to-Singlish transliteration 66

Table B.1: Use of AI and automated tools 68

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
F1	Harmonic mean of precision and recall
FP16	16-bit floating point (mixed precision)
FR	Functional Requirement
GPS	Global Positioning System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
NFR	Non-Functional Requirement
NLP	Natural Language Processing
PDPA	Personal Data Protection Act
REST	Representational State Transfer
RQ	Research Question
SDG	Sustainable Development Goal
SVM	Support Vector Machine
TC	Test Case
XLM-R	XLM-RoBERTa (Cross-lingual Language Model, Robustly Optimised BERT Approach)

1 INTRODUCTION

1.1 Overall project context

The informal household maintenance sector in Sri Lanka covers plumbers, electricians, carpenters, masons, painters, appliance technicians and many other trades. Most of these workers are self-employed, are found through word of mouth, and have no shared record of how well they have performed in the past. A customer who needs a leaking pipe fixed or a ceiling fan replaced usually asks neighbours or relatives for a phone number, or searches a classified-advertisement site that lists workers with little or no evidence of quality. Internet and smartphone use has grown steadily in the country; at the start of 2025 about 12.4 million people, or 53.6% of the population, used the internet [1]. A mobile marketplace that connects customers with suitable workers is therefore realistic, but such a marketplace is only useful if customers can trust the information it shows about each worker.

Research group R26-IT-090 is building an AI-powered smart service marketplace for household maintenance in Sri Lanka. The solution is a mobile application for customers and workers, supported by a Node.js backend, a MongoDB Atlas database and a set of Python FastAPI services that host the intelligent components. The project has four components:

- Component 1, Intent Recognition: the customer describes the problem in free text in English or Singlish, and a machine learning model predicts the type of worker that is needed.
- Component 2, Hybrid Worker Recommendation: a Random Forest model and a real-time mathematical model rank the available workers in the requested category, using qualifications, past completion time, live GPS distance and the worker's rating score, and return the top 20 candidates.
- Component 3, Automated Review Credibility and Sentiment Analysis (this thesis): after a job is completed, the customer's star rating and written review are analysed and combined into a category-specific trust score for the worker, which Component 2 uses as its rating input.
- Component 4, Fair Price Prediction: planned as the fourth component.

Figure 1.1 shows how the components fit together. The customer's request passes through Component 1 and Component 2 to produce a ranked list of workers. After the job is completed, the customer's review passes through Component 3, and the resulting trust score is stored and sent back to Component 2 so that the next ranking reflects the latest customer experience.

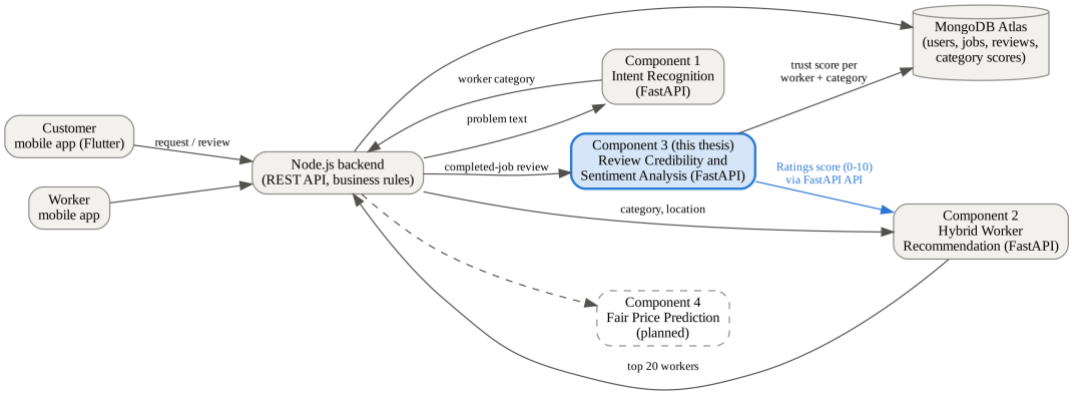


Figure 1.1: Overall architecture of the smart service marketplace and the position of Component 3

1.2 Individual component context

This thesis covers Component 3, Automated Review Credibility and Sentiment Analysis. The component is triggered when a customer finishes a job and leaves feedback. The feedback contains a star rating from one to five and, optionally, a short written review. Customers in Sri Lanka write such reviews in English, in Sinhala script, or in Singlish, which is Sinhala written informally with English letters and freely mixed with English words (for example, "wadak na ade mu", meaning roughly "useless work, man"). The component has four jobs:

1. Accept a review only when it belongs to a job that has been completed in the application, so that every review comes from a real customer of that worker.
2. Classify the sentiment of the written review as Negative, Neutral or Positive with a multilingual transformer model, XLM-RoBERTa (XLM-R) [2], fine-tuned for this task, and convert the prediction into a sentiment value between -1 and +1.
3. Combine the star ratings and the sentiment values of all reviews that the worker has received in one service category into a single score from 0 to 10, and adjust that score according to how many reviews it is based on.

4. Make the final category-specific trust score available to Component 2 through an application programming interface (API).

In this thesis the word credibility has a specific meaning. It is not the detection of fake reviews. Fake reviews are prevented by the business rule that only customers of completed jobs can write a review. Credibility means how much confidence the system can place in a worker's score, which depends on how many verified reviews the score is based on. A score built from one review is not credible; a score built from fifty reviews is. The component expresses this with a credibility factor $C = N/(N + 5)$, where N is the number of reviews in the category, and uses it to pull scores that rest on few reviews towards a neutral starting value (Section 4.4).

1.3 Problem statement

Online marketplaces rely heavily on star ratings, but star ratings alone are a weak signal of service quality. Three problems are relevant to a household-service marketplace in Sri Lanka.

First, star ratings and written reviews often disagree. A customer may select five stars by habit and then write a complaint, or select a low rating by mistake while praising the worker in the text. In a study of about 164,000 TripAdvisor hotel reviews, Fazzolari et al. found that the text and the score pointed in opposite directions for about 6% of reviews, and for about 17% of reviews with a score of 2 [3]. Rating distributions are also strongly skewed towards high values [4], [5], so an average of stars alone separates good and poor workers poorly.

Second, an average that is based on very few reviews is unreliable. A worker with one five-star review has an average of 5.0, which is higher than a worker with forty reviews averaging 4.8, even though the second worker's record is much stronger evidence of quality. Ride-hailing platforms acknowledge this; Uber, for example, notes that a new driver's average can swing noticeably over the first 50 to 100 rated trips [6]. If the marketplace's recommendation engine uses raw averages, a single lucky or unlucky review can move a worker to the top or bottom of the list.

Third, the written reviews in this market are multilingual and code-mixed. Customers write in English, Sinhala script and Singlish, often in the same sentence, with informal

spelling, slang and emoji. Most sentiment analysis tools are built for English, and Sinhala is a low-resource language for natural language processing (NLP) [7]. A sentiment model that only understands English would ignore or misread a large part of the feedback.

Some platforms also accept reviews from people who never used the service, which opens the door to fake or misleading reviews and reduces customer trust [8], [9].

The problem addressed by this component is therefore: how can a household-service marketplace turn verified customer feedback, consisting of star ratings and optional English, Sinhala or Singlish text, into a single, category-specific worker score that reflects both what customers rated and what they wrote, and that does not overreact when the score is based on only a few reviews?

1.4 Research gap

Sentiment analysis is a mature field for English product and movie reviews [10], [11], and multilingual transformers such as XLM-R have made it possible to classify text in many languages with one model [2], [12]. Work on Sinhala and Sinhala-English code-mixed text exists, but it mostly covers news comments, social media posts and general sentiment, uses Sinhala script or limited code-mixed corpora, and stops at the classification step [13], [14], [15], [16]. None of these studies turns the predicted sentiment into a worker reputation score.

Reputation and trust systems, on the other hand, have been studied for decades [17], [18], and practical methods for ranking items by average rating with few votes, such as Bayesian averages and confidence intervals, are well known [19], [20]. These methods operate on star ratings or up and down votes and do not use the content of the written review.

The gap that this component addresses lies between these two bodies of work. No published system for Sri Lankan household services (a) analyses English, Sinhala-script and Singlish review text with a single multilingual model, (b) fuses the resulting sentiment with star ratings, (c) adjusts the fused score for the number of reviews with a Bayesian prior, and (d) keeps the score separate for each service category so that, for

example, a poor electrical job does not reduce a worker's plumbing reputation. Section 2.5 develops this gap in more detail.

1.5 Aim and objectives

The aim of this component is to design, build and evaluate a review analysis service that produces a credible, category-specific trust score for each worker from verified star ratings and multilingual review text, and supplies that score to the worker recommendation engine.

The specific objectives are:

1. To prepare a labelled three-class (Negative, Neutral, Positive) sentiment dataset that contains English, Sinhala-script and Singlish text, by extending a public English dataset with machine translation, rule-based transliteration and template-generated Singlish sentences.
2. To fine-tune the multilingual transformer XLM-R on this dataset and evaluate it with accuracy, weighted F1, per-class precision, recall and F1, and a confusion matrix, targeting at least 85% accuracy on the held-out test set.
3. To design a scoring model that fuses star ratings and review sentiment into a 0 to 10 score, handles reviews that have no text, and adjusts the score for the number of reviews through a credibility factor and a Bayesian prior.
4. To keep each worker's score separate for every service category and to accept reviews only for completed jobs.
5. To implement the model and the scoring logic as a FastAPI service, store the results in MongoDB Atlas, and deliver the trust score to Component 2 through an API.

1.6 Research questions or hypotheses

The work is guided by the following research questions.

- RQ1: How accurately can a single fine-tuned multilingual transformer classify the sentiment of English, Sinhala-script and Singlish review text into Negative, Neutral and Positive classes?

- RQ2: How can star ratings and text sentiment be combined into one worker score so that disagreement between the two is reflected in the score?
- RQ3: How can the score be made reliable for workers who have only a few reviews, without hiding real differences between workers who have many?

For RQ1 the working hypothesis is that a fine-tuned XLM-R model can reach at least 85% accuracy and weighted F1 on the held-out test set, which would be comparable with published results for Sinhala and code-mixed sentiment [16]. RQ2 and RQ3 are design questions; they are answered by the scoring model in Section 4.4 and checked with worked scenarios in Section 6.2.

1.7 Scope limitations and assumptions

The component is in scope for the following:

- Three-class sentiment classification of review text in English, Sinhala script and Singlish.
- Fusion of star ratings and sentiment into a 0 to 10 trust score per worker and per service category, with a credibility adjustment based on the number of reviews.
- Accepting reviews only after job completion, storing the results in MongoDB Atlas, and delivering the score to Component 2 through an API.

The following are out of scope:

- Detecting fake, paid or bot-generated reviews with machine learning. This is handled by the verified-completion business rule instead.
- Aspect-based sentiment (for example, separate scores for punctuality, price and quality).
- Tamil and Tamil-English (Tanglish) reviews.
- Detecting sarcasm, and moderating offensive language in reviews.

The work makes the following assumptions:

- A customer can review a job only after the job is marked as completed in the application, and the job is linked to one worker and one service category.
- A review always has a star rating; the written text is optional.

- The sentiment patterns learned from the augmented Twitter dataset transfer reasonably to short service reviews. This assumption is examined critically in Section 7.5.

1.8 Individual contribution

The author was responsible for all parts of Component 3:

- Defining the component's role, inputs and outputs within the group's architecture, and agreeing the interface to Component 2.
- Preparing the training data: selecting the public Kaggle dataset, writing the augmentation scripts for Sinhala translation, rule-based Sinhala-to-Singlish transliteration and template-based Singlish slang, and building the final 101,217-row dataset.
- Fine-tuning and evaluating the XLM-R sentiment model, including the training notebook, checkpoint recovery, per-epoch reporting, per-class evaluation and smoke tests.
- Designing the trust-scoring model: the 60/40 fusion of star and sentiment scores, the handling of reviews without text, the credibility factor and the Bayesian prior, and category isolation.
- Implementing the FastAPI service that hosts the model and the scoring logic, connecting it to the Node.js backend and MongoDB Atlas, and exposing the score to Component 2.

The shared mobile application, the Node.js backend and the database schema were developed jointly by the group. The other three components were developed by other group members and are described here only where they interact with Component 3.

1.9 Thesis structure

Chapter 2 reviews the literature on sentiment analysis, Sinhala and code-mixed NLP, online reviews and reputation systems, and identifies the gap. Chapter 3 describes the research methodology, the dataset, the training procedure and the evaluation metrics. Chapter 4 presents the requirements and the design of the component, including the scoring model. Chapter 5 describes the implementation, the integration with the other components and the testing. Chapter 6 reports the results. Chapter 7 discusses the

findings, the contributions and the limitations. Chapter 8 covers commercialisation and sustainability. Chapter 9 concludes the thesis and sets out future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature for this chapter was gathered from the ACL Anthology, IEEE Xplore, the ACM Digital Library, SpringerLink, arXiv, the University of Moratuwa institutional repository and Google Scholar. The search terms combined three groups of keywords: (a) sentiment analysis, opinion mining, transformer, BERT, XLM-RoBERTa, multilingual; (b) Sinhala, Singlish, code-mixed, code-switched, transliteration, low-resource; and (c) online reviews, rating, reputation system, trust, Bayesian average, fake review, opinion spam. Sources were included when they were peer-reviewed or widely cited, and when they addressed sentiment classification, Sinhala or code-mixed text, or the reliability of online ratings. Platform documentation and industry reports were used only to describe how existing systems work. The review covers work from 2000 to 2025, with emphasis on transformer-based methods published after 2018.

The chapter is organised in four parts. Section 2.2 covers the theoretical foundations: sentiment analysis, transformer language models and the statistics of ratings. Section 2.3 reviews related research on Sinhala and code-mixed sentiment, review-rating disagreement and opinion spam. Section 2.4 describes how existing platforms present ratings. Section 2.5 brings these together into the research gap and the conceptual framework of the component.

2.2 Domain and theoretical foundations

2.2.1 Sentiment analysis

Sentiment analysis, or opinion mining, is the computational study of the opinions, attitudes and emotions expressed in text [21], [22]. At the document or sentence level it is usually treated as a classification task: a piece of text is assigned to a polarity class such as positive or negative, often with a third neutral class. Early work used lexicons of positive and negative words and hand-written rules. Lexicon methods such as VADER remain popular for social media because they need no training data and are easy to explain [23], but they depend on a word list for one language and cannot handle words that are not in the list, which is a serious problem for informally spelled Singlish.

Pang, Lee and Vaithyanathan showed that supervised machine learning methods, such as Naive Bayes, maximum entropy and support vector machines (SVMs) trained on bag-of-words features, outperform hand-built baselines on movie reviews [10]. Neural models later improved on these results by learning dense representations of words and sentences: convolutional networks [24], recursive networks over parse trees [11] and word vectors trained for sentiment [25]. Large review datasets from Yelp and Amazon became standard benchmarks for these models [26].

2.2.2 Transformers and multilingual language models

The transformer architecture replaced recurrence with self-attention, allowing every token in a sentence to attend directly to every other token [27]. BERT pre-trained a transformer encoder on large amounts of unlabelled text with a masked-language-model objective and then fine-tuned it for downstream tasks, setting new results across many classification benchmarks [28]. RoBERTa showed that training the same architecture longer, on more data and without the next-sentence objective gives further gains [29]. Sun et al. studied how to fine-tune BERT for text classification and found that small learning rates, around 2e-5, and a few epochs work well [30].

Multilingual models apply the same idea to many languages at once. Multilingual BERT was trained on Wikipedia in 104 languages and showed surprising cross-lingual transfer [31]. XLM-RoBERTa (XLM-R) was trained with the RoBERTa recipe on 2.5 TB of filtered CommonCrawl text in 100 languages, including Sinhala, and uses a SentencePiece tokenizer with a 250,000-token vocabulary shared across languages [2], [32]. Because SentencePiece splits unknown words into sub-word pieces, XLM-R can produce a representation for any input string, including romanised Singlish words that never appear in a dictionary. XLM-T, an XLM-R model further trained on about 198 million tweets, showed that this family of models works well for multilingual Twitter sentiment [12]. The base version of XLM-R has about 270 million parameters, which is small enough to fine-tune on a single graphics processing unit (GPU) and to serve on a modest cloud server.

Fine-tuning is usually done with the AdamW optimiser, which decouples weight decay from the gradient update [33], a short linear warm-up of the learning rate, and cross-

entropy loss over the output classes. The Hugging Face Transformers library provides standard implementations of these models and training loops [34].

2.2.3 Evaluation of classifiers

Classifier performance is usually reported with accuracy, precision, recall and the F1 score, the harmonic mean of precision and recall [35]. When classes are not perfectly balanced, macro-averaged F1 (the unweighted mean over classes) and weighted F1 (the mean weighted by class support) give a fuller picture than accuracy alone, and a confusion matrix shows which classes are mistaken for which. Two further issues matter for this work. Modern neural networks are often overconfident: the probability they assign to their prediction is higher than their real accuracy [36]. And when near-duplicate examples appear in both the training and test sets, the measured accuracy overstates the accuracy on genuinely new data, a form of data leakage [37]. Behavioural tests, in which hand-written inputs probe specific abilities of a model, complement aggregate metrics [38].

2.2.4 Ratings, reputation and Bayesian averaging

A reputation system collects feedback about the past behaviour of participants and makes it available to help others decide whom to trust [17]. Jøsang, Ismail and Boyd surveyed the computation models used by such systems, ranging from simple sums and averages to Bayesian models based on beta distributions [18]. Two statistical properties of ratings are important here.

The first is skew. Online ratings tend to follow a J-shaped distribution, with many five-star ratings, some one-star ratings and few in between, because people with extreme experiences are more likely to write a review [4]. On Airbnb, nearly 95% of properties have an average of 4.5 or 5 stars [5]. When almost everyone is rated highly, the average star rating carries little information on its own.

The second is sample size. The average of a small number of ratings has high variance. Ranking items by their raw average puts items with one perfect rating above items with hundreds of excellent ratings. Proposed fixes include ranking by the lower bound of a Wilson score confidence interval [39], [19], and Bayesian approaches in which

each item starts from a prior belief that is updated as ratings arrive [20], [40]. A common practical form is the Bayesian average:

$$\textit{Bayesian average} = (n \times \textit{mean of ratings} + m \times \textit{prior mean}) / (n + m)$$

where n is the number of ratings and m is the weight given to the prior, expressed as an equivalent number of ratings. With few ratings the result stays close to the prior mean; as n grows it moves towards the item's own average. This form is used in the scoring model of this component (Section 4.4).

2.3 Related research

2.3.1 Sinhala and code-mixed sentiment analysis

Sinhala is spoken by about 16 million people as a first language and is classed as a low-resource language for NLP. De Silva's survey of Sinhala NLP tools and research lists many small, often unavailable, datasets and tools, and very little work on code-mixed text [7]. Liyanage studied sentiment analysis of Sinhala news comments in Sinhala script [41]. Senevirathne et al. compared recurrent, long short-term memory (LSTM), hierarchical attention and capsule networks for four-class (positive, negative, neutral, conflict) sentiment of Sinhala news comments, and the best models reached a weighted accuracy of only about 63% [13], which shows how hard Sinhala sentiment remains without large pre-trained models.

Code-mixed text adds further difficulty. Words from two languages are mixed in one sentence, and one of the languages is often written in a non-native script. Benchmarks such as GLUECoS for Hindi-English and Spanish-English [42], the SemEval-2020 shared task on sentiment in code-mixed tweets [43] and the Tamil-English corpus of Chakravarthi et al. [44] show that code-mixed sentiment is consistently harder than monolingual sentiment. For Sinhala-English, Chathuranga and Ranathunga used capsule networks to classify code-mixed text for sentiment, humour, hate speech and aspect identification [15]. Rathnayake et al. fine-tuned multilingual models, including XLM-R, with adapter layers on code-mixed and code-switched Sinhala-English, Kannada-English and Hindi-English text, and found that adapter-based fine-tuning matches or beats full fine-tuning while training far fewer parameters [14]. Most recently, Rizvi et al. proposed a multilingual sentiment pipeline for Sinhala, English

and code-mixed content in which a fine-tuned XLM-R handles Sinhala and code-mixed text with reported performance of 88.4%, and BERT-base handles English with 92.3% accuracy [16].

Transliteration between Sinhala script and romanised Sinhala is closely related. De Mel et al. compared a rule-based transliterator with a fine-tuned sequence-to-sequence model for romanised Sinhala and found that the neural model was much more accurate on informal spellings, while the rule-based system was much faster [45]. This finding matters for the present work, which uses a rule-based transliterator in the opposite direction, from Sinhala script to romanised text, to create synthetic Singlish training data (Section 3.2).

These studies show that multilingual transformers, and XLM-R in particular, are the strongest available tools for Sinhala and code-mixed sentiment. They also show that labelled Singlish data is scarce, so researchers create or augment their own datasets. None of the studies applies the classifier to service reviews or uses its output to score service providers.

2.3.2 Data augmentation for low-resource text

When labelled data in the target language is scarce, augmentation can create additional training examples. Simple operations such as synonym replacement, random insertion and random swapping of words improve small text classifiers [46]. Back-translation, translating text into another language and back, was introduced for machine translation and is widely used to create paraphrases [47]. Translating an existing labelled dataset into the target language and keeping the original labels is a common way to bootstrap a classifier for a low-resource language. The risk is that translated text is more formal than real user text and that translation errors may change the meaning while the label stays the same.

2.3.3 Disagreement between review text and star rating

Several studies show that the star rating and the text of the same review do not always agree. Fazzolari et al. classified the polarity of about 164,000 TripAdvisor hotel reviews and found disagreement with the numeric score in about 6% of cases overall, rising to 17% for reviews with a score of 2 [3]. They argued that many such reviews

mix positive and negative points, so the text carries information that the score loses. McAuley and Leskovec showed that combining the text of reviews with ratings improves rating prediction [48]. Mudambi and Schuff found that the depth of a review and the extremity of its rating affect how helpful readers find it [49]. These results support using both signals instead of either one alone.

2.3.4 Fake reviews and opinion spam

Fake reviews are a known problem. Jindal and Liu first studied opinion spam on Amazon [50]. Ott et al. built a dataset of deceptive hotel reviews written by paid crowd workers and showed that humans detect them at little better than chance, while a supervised classifier reached about 90% accuracy [8]. Luca found that a one-star increase in a restaurant's Yelp rating was associated with a 5% to 9% increase in revenue, which explains why businesses have an incentive to post fake reviews [9]. Detecting fake reviews with machine learning is a difficult research problem in its own right. A simpler and stronger defence, used by many service platforms, is to allow a review only from a customer who has completed a verified transaction. This component takes that approach.

2.4 Existing systems products and techniques

Table 2.1 summarises how representative platforms collect and present ratings. Most platforms show an average star rating and the number of ratings, and display written reviews separately for the customer to read. The written text is not used to compute the displayed score.

Table 2.1: Rating approaches of existing platforms and techniques

Platform or technique	Who can rate	How the score is computed	Uses review text in the score	Handles few ratings
Uber (ride hailing)	Riders after a completed trip	Rolling average of the most recent 500 ratings [6]	No	No adjustment; average is noted to be unstable for new drivers
Airbnb (accommodation)	Guests after a completed stay	Average of star ratings, shown with review count [5]	No	No; scores are compressed near 5

Platform or technique	Who can rate	How the score is computed	Uses review text in the score	Handles few ratings
Classified-advertisement sites for services	Often no rating, or any visitor	Listing only, or simple average	No	No
Bayesian average or Wilson lower bound	Any rating source	Average shrunk towards a prior, or confidence bound [19], [20]	No	Yes
Text-based sentiment models	Any text source	Polarity of each review [16], [12]	Yes	Not addressed; no reputation score
This component	Customers of completed jobs only	60% stars, 40% sentiment, Bayesian credibility adjustment, per category	Yes (English, Sinhala, Singlish)	Yes

Verified-transaction platforms such as Uber and Airbnb show that the "review only after completion" rule is accepted practice. However, their scores rest on stars alone, and the heavy concentration of ratings near five stars makes it hard to tell good workers from excellent ones [5]. Bayesian averaging is common in product and media ranking, but it is applied to stars or votes only. Sentiment models produce a polarity for each text but do not turn it into a reputation score. No platform reviewed combines the three ideas, and none supports Singlish reviews.

2.5 Gap synthesis and conceptual framework

The review supports four observations:

1. Star ratings alone are a weak and skewed signal, and they disagree with the review text often enough to matter [3], [4].
2. Averages based on few ratings are unreliable, and Bayesian shrinkage towards a prior is an established remedy [20], [40].
3. Multilingual transformers such as XLM-R are the most effective available approach to Sinhala and code-mixed sentiment, but Singlish training data is scarce and no work applies them to service reviews [2], [14], [16].
4. Verified-completion rules are an effective, simple defence against fake reviews and are already common practice.

The gap is a scoring method that brings these together for the Sri Lankan household-service setting: a verified, multilingual, text-aware and sample-size-aware score, computed separately for each service category. Figure 2.1 shows the conceptual framework of the component. Verified job completion guarantees the integrity of the inputs. The star rating gives a quantitative signal and the text, analysed by a multilingual transformer, gives a qualitative one. The two are fused with fixed weights, then adjusted for the number of reviews through a Bayesian credibility factor, separately for each service category. The resulting trust score is used by Component 2 to rank workers.

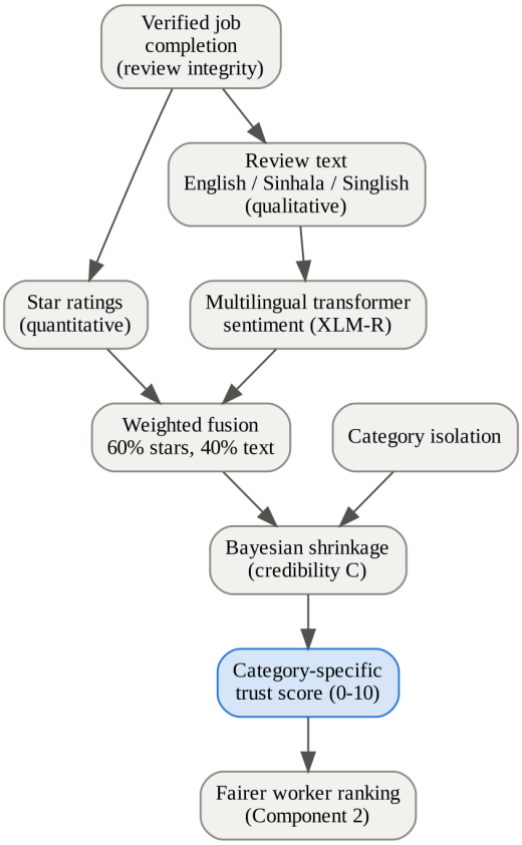


Figure 2.1: Conceptual framework of the review credibility and sentiment component

3 RESEARCH METHODOLOGY

3.1 Research design

The component was developed with a design-and-evaluate approach that combines an experimental study with the construction of a software artefact. The research has two strands that match the two halves of the component.

The first strand is a quantitative experiment that answers RQ1. A pre-trained multilingual transformer is fine-tuned on a labelled sentiment dataset, and its performance is measured on a held-out test set with standard classification metrics. The independent variables are the training data and the training settings; the dependent variables are accuracy, precision, recall and F1. The experiment also includes a small set of hand-written behavioural tests in English, Sinhala and Singlish.

The second strand is a design study that answers RQ2 and RQ3. A scoring model is designed from principles established in the literature (fusion of rating and text signals, and Bayesian shrinkage), implemented in the FastAPI service, and checked with worked scenarios that cover typical and edge cases, such as a new worker, a worker with one review, a worker whose reviews have no text, and a worker whose stars and text disagree.

The work followed an iterative cycle. An earlier version of the plan used a lightweight binary classifier (Naive Bayes or SVM) that would label each review as positive or negative and compute the percentage of positive reviews. As the work progressed, this was replaced by a three-class transformer model and a 0 to 10 scoring model, for the reasons given in Section 4.3. Figure 3.1 shows the overall research and development process.

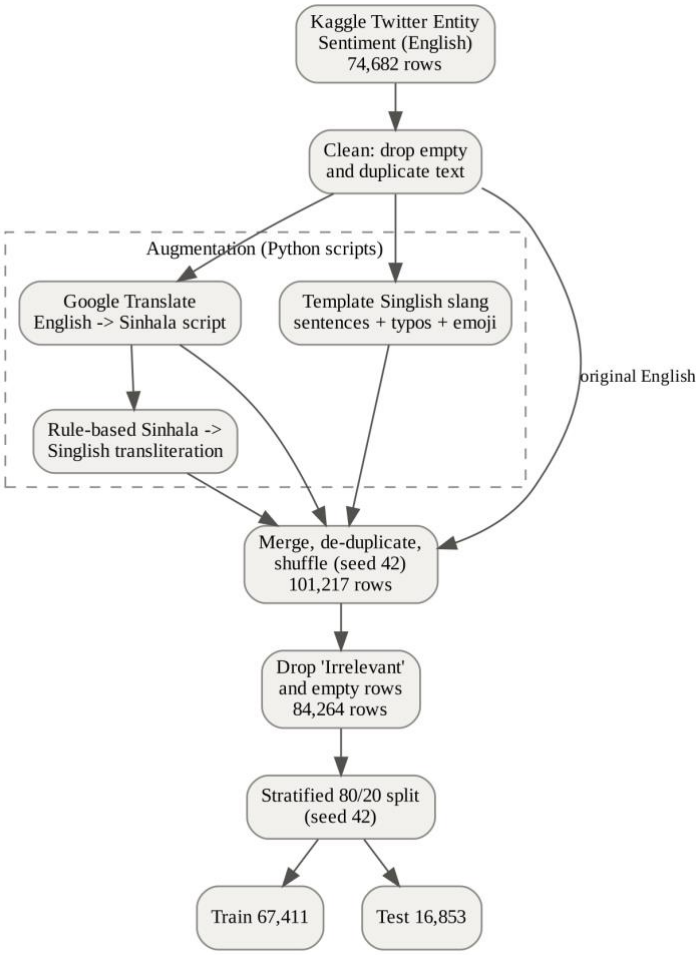


Figure 3.1: Data preparation and model training pipeline

3.2 Data participants or study materials

3.2.1 Base dataset

No public dataset of Sri Lankan household-service reviews exists, and the marketplace application had no real reviews when the model was trained. The base dataset was therefore the Twitter Entity Sentiment Analysis dataset published on Kaggle [51]. Its training file, twitter_training.csv, contains 74,682 rows with four columns: a tweet identifier, the entity (topic) that the tweet is about, a sentiment label, and the tweet text. The labels are Positive, Negative, Neutral and Irrelevant. The tweets are in English and discuss brands, video games and other entities. This dataset was chosen because it is large, it is labelled with three usable sentiment classes plus a fourth class that can be removed, and its short, informal social media text is closer in style to customer reviews written on a phone than formal product reviews are.

3.2.2 Augmentation with Sinhala and Singlish text

To teach the model Sinhala and Singlish, the English dataset was extended with three kinds of generated text. All three keep the label of the English tweet they were derived from. The augmentation was written in Python and run with 20 parallel worker threads to speed up the translation calls.

1. Sinhala machine translation. Each English tweet was translated into Sinhala script with Google Translate, called through the deep-translator Python library (source language detected automatically, target language Sinhala). Rows for which translation failed were skipped.
2. Rule-based Sinhala-to-Singlish transliteration. The Sinhala translation was converted into romanised Singlish with a transliteration function written for this project. The function maps each Sinhala consonant, independent vowel and vowel sign (pili) to Latin letters, for example ත to "th", ද to "dh", ශ and ෂ to "sh", and ව to "w". It adds the inherent vowel "a" after a consonant unless a vowel sign or the virama (hal kirima) follows. It also handles the zero-width joiner and the conjunct forms rakaransaya (්‍ර, giving "r") and yansaya (්‍ය, giving "y"). A final step collapses repeated vowel letters. For example, "මෙය ඉතා හොඳයි" becomes "meya ithaa hondayi", and "ප්‍රශ්නයක් නැහැ" becomes "prashnayak naehae".
3. Template Singlish slang sentences. Short Singlish sentences were generated from templates filled with slang words of the right polarity. The word lists contain 10 positive words (for example "supiri", "patta", "niyamai"), 10 negative words (for example "maru", "karadarai", "awul") and 6 neutral words. Each word is randomly given a typing variation (repeated final letters, doubled vowels or capital letters), placed in one of 8 positive, 7 negative or 4 neutral templates, and followed by a matching emoji with 70% probability. These sentences imitate the casual way Singlish is typed on phones.

After augmentation, the original English rows and all generated rows were merged, rows with duplicate text were removed, and the dataset was shuffled with a fixed random seed of 42. The final file, twitter_training_fast_augmented.csv, contains 101,217 rows. Its label distribution is shown in Figure 3.2 and Table 3.1.

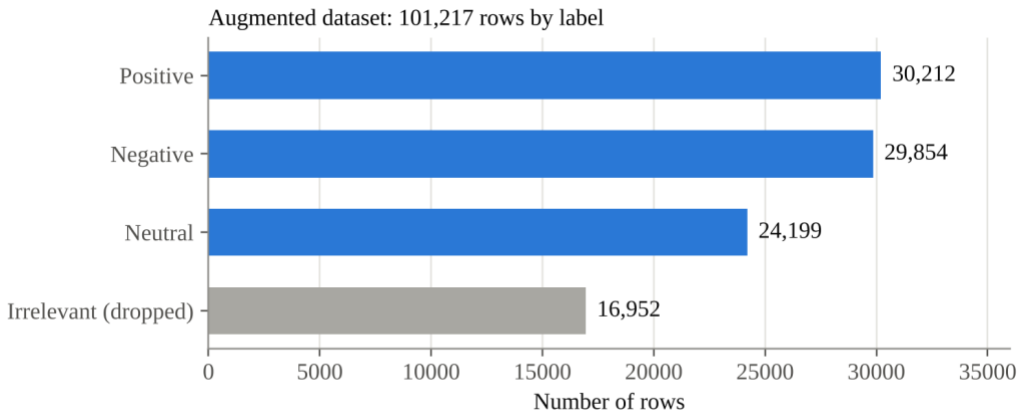


Figure 3.2: Label distribution of the augmented dataset

Table 3.1: Label distribution of the augmented dataset

Label	Rows	Share of dataset	Used for training
Positive	30,212	29.8%	Yes
Negative	29,854	29.5%	Yes
Neutral	24,199	23.9%	Yes
Irrelevant	16,952	16.7%	No (removed)
Total	101,217	100.0%	84,264 rows after cleaning

A further augmentation step was attempted while preparing the final training run. It sampled 2,500 English-only rows and aimed to add 5,000 new Sinhala and Singlish rows. In the attempted run, every translation request failed, most likely because of rate limiting by the translation service when 25 requests were sent in parallel, and no rows were added. This step was therefore switched off, and the final model was trained on the 101,217-row dataset as described above.

The number of rows of each language type (English, Sinhala script, transliterated Singlish and template Singlish) was not recorded separately when the dataset was built. This is a limitation for the per-language analysis and is discussed in Section 7.5.

3.2.3 Data cleaning and label encoding

Before training, only the text and sentiment columns were kept, and rows with an empty value in either column were dropped. The three sentiment labels were encoded as integers: Negative = 0, Neutral = 1 and Positive = 2. Rows labelled Irrelevant have no mapping and were removed, because "irrelevant" describes whether a tweet is about

the named entity, not the sentiment of the text, and has no equivalent in a service review. After cleaning, 84,264 rows remained.

No other text preprocessing was applied. The text was not lower-cased, stemmed or stripped of emoji, because the XLM-R tokenizer is designed to work with raw text and the emoji and spelling variations in Singlish carry sentiment.

3.2.4 Train and test split

The cleaned data was split into a training set (80%) and a test set (20%) with scikit-learn's train_test_split [52], stratified by label so that both sets have the same class proportions, with a random seed of 42. Table 3.2 shows the result.

Table 3.2: Training and test sets

Set	Negative	Neutral	Positive	Total
Training	23,883	19,358	24,170	67,411
Test	5,971	4,840	6,042	16,853
Total	29,854	24,198	30,212	84,264

The Neutral total in Table 3.2 is one row lower than in Table 3.1 because one Neutral row had empty text and was removed during cleaning.

3.3 Experimental or study procedure

3.3.1 Training environment

The final model was trained in Google Colab on an NVIDIA Tesla T4 GPU with about 15 GB of memory, using Python 3, PyTorch 2.11.0 [53] and Hugging Face Transformers 5.18.0 [34]. Checkpoints and the final model were saved to Google Drive so that training could resume from the last completed epoch if the Colab session disconnected. The notebook detects the available GPU memory and chooses the batch size automatically: with more than 6 GB it uses a batch size of 32 with no gradient accumulation; with less it uses a batch size of 4 with 8 accumulation steps, and on a central processing unit (CPU) a batch size of 2 with 16 accumulation steps. On the T4, the first setting was used. An earlier version of the same notebook was run on Kaggle with a T4 GPU.

3.3.2 Model and tokenisation

The base model was xlm-roberta-base [2], loaded with a sequence-classification head of three outputs. The classification head (a dense layer and an output projection) is newly initialised, while the encoder starts from the pre-trained weights. Each text was tokenised with the model's SentencePiece tokenizer [32], truncated or padded to 128 tokens. A length of 128 tokens covers tweets and typical short reviews.

3.3.3 Training settings

Table 3.3 lists the training settings. The learning rate, weight decay and warm-up follow common practice for fine-tuning BERT-style models [30], [33].

Table 3.3: Training hyperparameters

Setting	Value
Base model	xlm-roberta-base (about 270 million parameters)
Number of classes	3 (Negative, Neutral, Positive)
Maximum sequence length	128 tokens
Optimiser	AdamW (Transformers Trainer default)
Learning rate	2 × 10^-5 with linear decay
Warm-up	10% of training steps
Weight decay	0.01
Batch size	32 (training and evaluation)
Gradient accumulation	1
Mixed precision	FP16
Epochs	10
Steps per epoch / total steps	2,107 / 21,070
Evaluation and checkpointing	At the end of every epoch
Model selection	Checkpoint with the highest weighted F1 on the evaluation set
Random seed for split	42

3.3.4 Procedure

The procedure was as follows:

1. Load the augmented dataset, clean it, encode the labels and split it as in Section 3.2.
2. Tokenise both sets and load the pre-trained model.
3. Train for 10 epochs with the Hugging Face Trainer. At the end of each epoch, compute the loss, accuracy and weighted F1 on the test set and save a checkpoint.

4. After training, reload the checkpoint with the highest weighted F1, evaluate it again, and save it with its tokenizer as the final model.
5. Produce the per-epoch table and training curves, a per-class classification report and a confusion matrix on the test set.
6. Run a smoke test of nine hand-written sentences, one positive, one negative and one neutral in each of English, Sinhala script and Singlish.
7. Load the saved model in an interactive prediction loop and test further Singlish sentences by hand.

Training took 80 minutes on the T4 GPU.

3.4 Measures and evaluation metrics

The sentiment model was evaluated with the following metrics, all computed with scikit-learn [52]:

- Accuracy: the proportion of test reviews whose predicted label equals the true label.
- Precision, recall and F1 for each class. For a class k, precision is the share of reviews predicted as k that are truly k, recall is the share of reviews truly k that are predicted as k, and F1 = 2 × precision × recall / (precision + recall) [35].
- Macro F1 (the unweighted mean of the per-class F1 values) and weighted F1 (the mean weighted by the number of test reviews in each class).
- The confusion matrix, which counts how many reviews of each true class were assigned to each predicted class.
- Training and validation cross-entropy loss per epoch, to detect overfitting.
- Prediction confidence: the softmax probability of the predicted class, reported for the smoke tests.

The scoring model is deterministic, so it was evaluated by checking that it behaves as intended on worked scenarios: the score of a new worker, the effect of the number of reviews, the handling of reviews without text, and the effect of disagreement between stars and text (Section 6.2).

3.5 Data analysis

Per-epoch metrics were collected from the Trainer's log history and tabulated, and training curves were plotted with Matplotlib. The per-class report and the confusion matrix were computed on the predictions of the selected model for all 16,853 test reviews. The smoke-test predictions were examined one by one to see which language and class combinations the model handles well or badly. For the scoring model, a reference implementation of the formulas was used to compute the raw and final scores of eight scenarios, and the results were compared with the intended behaviour.

3.6 Ethics privacy security and responsible AI

The base dataset is publicly available on Kaggle and contains public tweets with no names or contact details beyond what appears in the tweet text; it was used only to train the model. No personal data of real customers or workers was used to train or evaluate the model, and no human participants took part in the study.

In the deployed application, reviews are personal data: they are linked to a customer, a worker and a job. The design follows the principles of Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [54]: reviews are collected for the stated purpose of rating workers, they are stored in a managed cloud database (MongoDB Atlas) rather than on individual devices, and only the aggregated trust score, not the reviewer's identity or review text, is passed to Component 2. A full compliance review, including consent wording and data retention periods, is needed before public launch (Section 8.4).

The scoring model affects workers' livelihoods, because it influences their position in the recommendation list. Several design choices reduce the risk of unfair outcomes:

- Only verified customers of completed jobs can review, which blocks reviews from competitors or people who never used the service.
- A new worker starts at a neutral 6.0 out of 10 instead of 0, and the effect of the first few reviews is limited by the credibility factor, so a single unfair review cannot destroy a new worker's reputation.
- Scores are kept separate for each service category, so a poor review in one trade does not reduce the worker's score in another.

- The formula is transparent and can be explained to workers: it is a fixed weighted average with a published adjustment for the number of reviews.

The sentiment model has known weaknesses, including overconfident predictions and lower accuracy on neutral sentences (Section 6.1). These are documented in this thesis in the spirit of model cards [55] and datasheets for datasets [56], so that the limits of the model are clear to its users.

3.7 Reproducibility and research artefacts

The research produced the following artefacts:

- The augmentation scripts for Sinhala translation, Sinhala-to-Singlish transliteration and template Singlish generation.
- The augmented dataset twitter_training_fast_augmented.csv (101,217 rows).
- The Colab training notebook, which contains the data loading, training, per-epoch reporting, per-class evaluation, smoke tests and interactive prediction.
- The fine-tuned model and tokenizer (the final_model folder), with the per-epoch metrics file, the training-curve figure, the classification report and the confusion matrix.
- The FastAPI service that loads the model and computes the trust score.

The split uses a fixed random seed (42), the dataset shuffle uses the same seed, and all library versions are recorded in the notebook output. Two sources of variation remain: the random initialisation of the classification head was not fixed with a seed (the model is created before the Trainer sets its default seed), and the Google Translate output used during augmentation may change over time. Re-running the notebook on the saved dataset should therefore give very similar but not identical results.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements were derived from the project problem, the role that the group assigned to Component 3, and the input that Component 2 needs. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements.

Table 4.1: Functional requirements of Component 3

ID	Requirement
FR1	The system shall accept a review only for a job whose status is completed.
FR2	Each review shall contain a star rating from 1 to 5 and may contain written text.
FR3	The system shall classify review text in English, Sinhala script or Singlish as Negative, Neutral or Positive, with a confidence value.
FR4	The system shall convert each classified review into a sentiment value between -1 and +1.
FR5	The system shall compute a worker's trust score from all of the worker's reviews in one service category, combining star ratings and sentiment values.
FR6	When none of the worker's reviews in a category has text, the score shall be computed from the star ratings alone.
FR7	The score shall be adjusted for the number of reviews, so that a worker with few reviews stays close to a neutral starting value.
FR8	A worker with no reviews in a category shall receive the neutral starting score of 6.0.
FR9	Scores for different service categories of the same worker shall be computed and stored separately.
FR10	The final trust score (0 to 10) shall be stored in the database and made available to Component 2 through an API.

Table 4.2: Non-functional requirements of Component 3

ID	Category	Requirement
NFR1	Accuracy	The sentiment model should reach at least 85% accuracy and weighted F1 on the held-out test set.
NFR2	Multilingu ality	One model should handle English, Sinhala script and Singlish without a separate language-detection step.
NFR3	Transparen cy	The scoring formula should be simple enough to explain to workers and customers.
NFR4	Fairness	A single review should not be able to move a new worker to the top or bottom of the ranking.
NFR5	Interoperab ility	The component should expose its functions as a REST API with JSON messages, so that the Node.js backend and Component 2 can call it.
NFR6	Maintainab ility	The model and the scoring constants should be replaceable without changing the rest of the system.
NFR7	Privacy	Only aggregated scores, not reviewer identities or review text, should be shared with Component 2.

4.2 Component architecture within the complete solution

Figure 1.1 showed the overall system. The customer and worker mobile applications talk to a Node.js backend, which holds the business logic (users, jobs and their status, and reviews) and stores data in MongoDB Atlas. The machine learning components run as separate Python FastAPI services. This split keeps the heavy Python libraries (PyTorch and Transformers) out of the Node.js backend and lets each component be developed, deployed and scaled on its own, which is the microservices approach the group chose for the project.

Component 3 is a FastAPI service with two parts:

1. The sentiment model: the fine-tuned XLM-R model and its tokenizer, loaded once when the service starts and kept in memory.
2. The scoring logic: the functions that turn reviews into a trust score, placed in the same service next to the model, so that a new review can be classified and the score updated in one step.

When a customer submits a review for a completed job, the backend passes it to Component 3. Component 3 classifies the text if there is any, stores the review's star rating and sentiment value under the worker and the job's service category, recomputes the worker's trust score for that category from all of the worker's reviews in the category, and saves the score. Component 2 obtains the score through the FastAPI API and uses it as its "Ratings score" input when ranking workers. Figure 4.1 shows the processing flow.

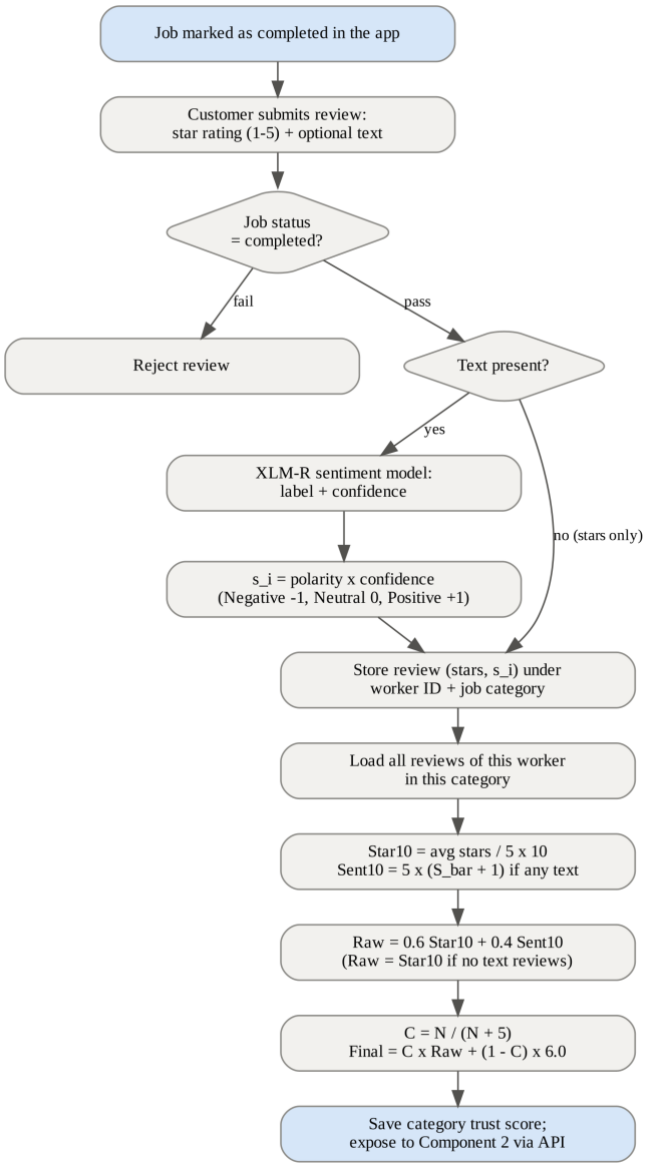


Figure 4.1: Review processing flow in Component 3

4.3 Design alternatives and decisions

Table 4.3 summarises the main design decisions and the alternatives that were considered.

Table 4.3: Design alternatives and decisions

Decision	Alternatives considered	Choice and reason
Sentiment classifier	Lexicon method (e.g. VADER); Naive Bayes or SVM on bag-of-words; multilingual transformer	Fine-tuned XLM-R. Lexicons and bag-of-words models need a fixed vocabulary and cannot cope with the many spellings of Singlish words. XLM-R's sub-word tokenizer and pre-training on 100 languages, including Sinhala, give one model for all three input types [2], [14].
Number of classes	Binary (Positive, Negative); three classes	Three classes. Many reviews are factual or mixed ("came on time, fixed the tap"). Forcing them into Positive or Negative distorts the score; a Neutral class maps naturally to a sentiment value of 0.
Model size	xlm-roberta-large; xlm-roberta-base	Base. It trains in about 80 minutes on a free T4 GPU and can be served on a modest server; the large model needs much more memory and time.
Text score of a worker	Percentage of positive reviews (original plan); mean sentiment value	Mean sentiment value. A percentage throws away the difference between a strongly and a weakly positive review and has no place for neutral reviews.
Sentiment value of one review	Fixed value per label (-1, 0, +1); label value multiplied by confidence	Label value multiplied by confidence. An uncertain prediction affects the score less than a confident one.
Score scale	0 to 100 percentage; 0 to 10	0 to 10. It is easy to read, it maps directly from five stars (×2), and it is the scale agreed with Component 2.
Fusion of stars and text	Stars only; text only; fixed weighted average	Fixed weights of 60% stars and 40% text. Stars are always present and are the customer's own explicit judgement; text adds information when it is available.
Few reviews	Raw average; minimum review count before showing a score; Wilson lower bound; Bayesian average	Bayesian average with a credibility factor. It gives every worker a score from the start, needs no threshold, and is easy to explain [20].
Fake reviews	Machine learning fake-review detector; verified completion	Verified completion. It is simple, robust and already standard practice; ML detectors need labelled fake reviews that do not exist for this market [8].
Where the formula runs	Node.js backend; FastAPI service	FastAPI service, next to the model, so that classification and scoring happen in one place.

The weights (60/40), the prior mean (6.0) and the prior weight (5) were chosen by the team as a fair balance, not tuned on data, because no real review data was available to tune them. The reasoning was as follows. Stars receive the larger weight because every review has them and they are the customer's direct rating, while the sentiment model

can make mistakes. The text still receives a substantial 40% so that a clear complaint in the text lowers a high star rating noticeably. The prior mean of 6.0 is slightly above the middle of the scale: a new worker is treated as acceptable but unproven, below the score that a worker with a good record earns. A prior weight of 5 means that the worker's own reviews and the prior count equally once the worker has five reviews, and that the worker's own record dominates after about 20 reviews (C = 0.8). Tuning these constants on real data is proposed as future work (Section 9.4).

4.4 Detailed design

4.4.1 Sentiment value of a review

For a review i that has text, the model outputs a probability for each of the three classes. The predicted label is the class with the highest probability, and its probability is the confidence p_i. The sentiment value of the review is:

$$s_i = \text{polarity}(\text{label}_i) \times p_i, \quad \text{where polarity(Negative)} = -1, \text{polarity(Neutral)} = 0, \text{polarity(Positive)} = +1$$

So s_i lies between -1 and +1. A confident positive review (p = 0.99) gives s_i = +0.99, an uncertain positive review (p = 0.60) gives +0.60, and every neutral review gives 0.

4.4.2 Star score and sentiment score

Let a worker have N reviews in a given service category, with star ratings r_1, ..., r_N, and let T of these reviews have text, with sentiment values s_1, ..., s_T. The star score and the sentiment score are both placed on a 0 to 10 scale:

$$\text{Star10} = (\text{mean of } r) / 5 \times 10$$

$$\bar{S} = (s_1 + s_2 + \ldots + s_T) / T, \quad \text{Sent10} = 5.0 \times (\bar{S} + 1.0)$$

Star10 ranges from 2.0 (all one-star) to 10.0 (all five-star). Sent10 ranges from 0 (every review confidently negative) to 10 (every review confidently positive), and equals 5.0 when the text is neutral on average.

4.4.3 Fusion

The raw score combines the two signals with fixed weights:

$$\text{Raw} = 0.60 \times \text{Star10} + 0.40 \times \text{Sent10} \quad (\text{if } T \geq 1)$$

$$Raw = Star10 \quad (if\ T = 0,\ i.e.\ no\ review\ in\ the\ category\ has\ text)$$

The second case follows the rule agreed for star-only feedback: when customers give stars without writing anything, the worker is rated on stars alone and no sentiment score is used. When some reviews have text and others do not, all N star ratings contribute to Star10 and the T texts contribute to Sent10.

4.4.4 Credibility and Bayesian adjustment

The credibility factor expresses how much the raw score can be trusted given the number of reviews:

$$C = N / (N + 5)$$

The final trust score is a weighted average of the raw score and the prior mean of 6.0:

$$Final = C \times Raw + (1 - C) \times 6.0$$

Substituting C shows that this is the Bayesian average of Section 2.2.4, with the prior counted as five reviews of score 6.0:

$$Final = (N \times Raw + 5 \times 6.0) / (N + 5)$$

A worker with no reviews has C = 0 and a final score of exactly 6.0. As N grows, C approaches 1 and the final score approaches the raw score. Figure 4.2 shows C as a function of N and the path of the final score for three workers with different raw scores. With 5 reviews C = 0.50, with 10 reviews C = 0.67, with 20 reviews C = 0.80 and with 50 reviews C = 0.91.

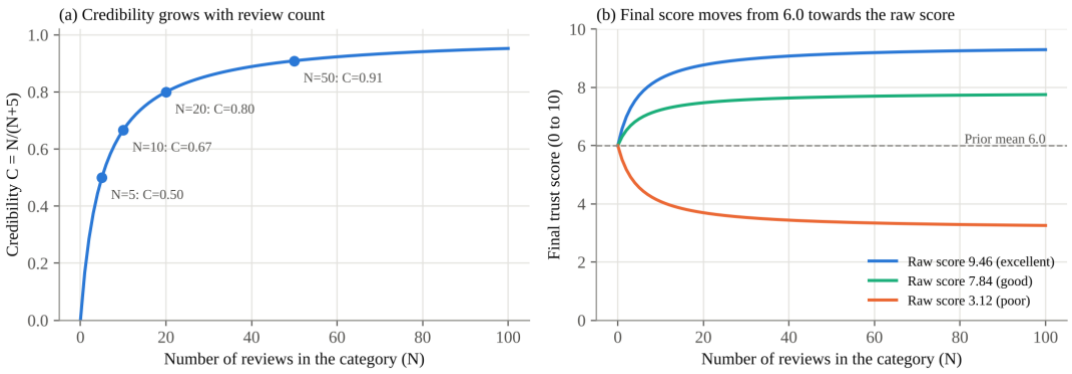


Figure 4.2: Credibility factor and its effect on the final trust score

4.4.5 Worked example

Consider a plumber with N = 10 reviews in the plumbing category. The average star rating is 4.2, and the mean sentiment value of the written reviews is $\bar{S} = +0.40$.

$$Star10 = 4.2 / 5 \times 10 = 8.40$$

$$Sent10 = 5.0 \times (0.40 + 1.0) = 7.00$$

$$Raw = 0.60 \times 8.40 + 0.40 \times 7.00 = 5.04 + 2.80 = 7.84$$

$$C = 10 / (10 + 5) = 0.667$$

$$Final = 0.667 \times 7.84 + 0.333 \times 6.0 = 7.23$$

The text pulls the score below what the stars alone suggest (8.40), because the written reviews are only mildly positive, and the credibility adjustment pulls it further towards 6.0 because ten reviews are still a modest amount of evidence.

4.4.6 Category isolation

Every review is stored with the worker's identifier and the service category of the job. The score is always computed from the reviews of one worker in one category. If a worker has done ten plumbing jobs and one electrical job, the plumbing score uses ten reviews and the electrical score uses one, and a poor electrical review does not affect the plumbing score. Because Component 2 ranks workers within the category requested by the customer, it always receives the score for that category.

4.4.7 Data design

The design stores two kinds of information in MongoDB Atlas. The first is the review record: the job, the worker, the customer, the service category, the star rating, the optional text, the predicted label, the confidence and the sentiment value. The second is the category score record for each worker and category: the number of reviews, the average star rating, the mean sentiment value, the raw score, the credibility factor and the final trust score. Storing the intermediate values as well as the final score makes it possible to explain a score and to recompute all scores if the weights are changed.

4.4.8 Service interface

The FastAPI service exposes two logical operations, both using JSON over HTTP:

- Analyse a review: receives the review (worker, category, star rating and optional text), runs the sentiment model when text is present, stores the review and returns the label, confidence, sentiment value and the updated category score.
- Get a trust score: receives a worker and a category and returns the final trust score (0 to 10), together with the number of reviews and the credibility factor. Component 2 uses this operation.

4.5 Standards and professional practice

The design follows several standards and accepted practices:

- Software quality: the requirements in Table 4.2 cover the ISO/IEC 25010 quality characteristics most relevant to the component: functional suitability, performance efficiency, compatibility, maintainability and security [57].
- Interfaces: the services communicate through REST-style HTTP endpoints with JSON payloads. FastAPI generates OpenAPI documentation for every endpoint automatically, which gives the other components a precise contract [58].
- Text encoding: all text is handled as Unicode (UTF-8), which is required for Sinhala script.
- Data protection: the design follows the purpose-limitation and data-minimisation principles of the Personal Data Protection Act of Sri Lanka [54].
- Responsible machine learning: the model's training data, metrics and known weaknesses are documented (Sections 3.2, 6.1 and 7.5), following the idea of model cards [55].
- Reproducibility: random seeds, library versions and hyperparameters are recorded in the training notebook (Section 3.7).

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Table 5.1 lists the tools and technologies used to build Component 3 and the parts of the shared system that it depends on.

Table 5.1: Technology stack

Layer	Technology	Use in Component 3
Model training	Google Colab with NVIDIA Tesla T4 GPU (final run); Kaggle Notebooks with T4 GPU (earlier runs)	Fine-tuning and evaluating XLM-R
Language and libraries	Python 3; PyTorch 2.11.0; Hugging Face Transformers 5.18.0 and Datasets; scikit-learn; pandas; NumPy; Matplotlib	Data handling, training, metrics and charts
Data augmentation	deep-translator (Google Translate); custom transliteration and template code	Creating Sinhala and Singlish training text
Model storage	Google Drive	Checkpoints and the final model during training
Model and scoring service	FastAPI (Python)	Serving the sentiment model and computing trust scores [58]
Backend	Node.js	Users, jobs, job status and reviews; calls to the AI services
Database	MongoDB Atlas	Reviews and category trust scores [59]
Client	Flutter mobile application	Review screen for customers
Integration	REST APIs with JSON	Backend to Component 3; Component 3 to Component 2

5.2 Component implementation

5.2.1 Data augmentation scripts

The augmentation scripts read the Kaggle file, remove rows with empty or duplicate text, and process the rows in parallel with a thread pool of 20 workers. For each row, the translation function calls Google Translate through deep-translator, passes the Sinhala result to the transliteration function, and builds new rows that copy the identifier, topic and sentiment label of the original. The template generator produces Singlish slang sentences for the same label. All new rows are added to the original data, duplicates are removed and the result is shuffled with seed 42 and written to twitter_training_fast_augmented.csv.

Listing 5.1 shows the core of the transliteration function. It walks through the Sinhala string one character at a time, looks up consonants, vowels and vowel signs in three dictionaries, and handles the virama and the conjuncts for "r" and "y".

Listing 5.1: Core of the Sinhala-to-Singlish transliteration function (abridged)

```
def sinhala_to_singlish(text):
    result, i, n = [], 0, len(text)
    while i < n:
        char = text[i]
        if char in consonants:
            base = consonants[char]                # e.g. 'ත' -> 'th'
            nxt = i + 1
            if nxt < n and text[nxt] == '':  # zero-width joiner
                nxt += 1
            if nxt < n and text[nxt] == '්':       # virama (hal kirima)
                after = nxt + 1
                if after < n and text[after] == '':
                    after += 1
                if after < n and text[after] == 'ර':   # rakaransaya
                    base += "r"; i = after; nxt = i + 1
                elif after < n and text[after] == 'ය': # yansaya
                    base += "y"; i = after; nxt = i + 1
            if nxt < n and text[nxt] in diacritics:  # vowel sign
                result.append(base + diacritics[text[nxt]]); i = nxt
            else:
                result.append(base + 'a')            # inherent vowel
        elif char in vowels:
            result.append(vowels[char])
        elif char in diacritics:
            result.append(diacritics[char])
        else:
            result.append(char)                      # spaces, digits,
Latin
        i += 1
    singlish = "".join(result)
    return singlish.replace("aaa", "aa").replace("eea", "ee")  # tidy-up
```

5.2.2 Training notebook

The training notebook is organised into numbered cells: package installation, a GPU check, mounting Google Drive, loading the dataset (from Kaggle, or by manual upload if the Kaggle download fails), the optional augmentation cell, a reporting helper, training, per-class evaluation and interactive prediction. Listing 5.2 shows the training configuration.

Listing 5.2: Training configuration in the notebook (abridged)

```
label_map = {"Negative": 0, "Neutral": 1, "Positive": 2}
df["label"] = df["sentiment"].map(label_map)
df = df.dropna(subset=["label"])                  # removes "Irrelevant"
train_df, test_df = train_test_split(df, test_size=0.2,
                                     random_state=42,
stratify=df["label"])
tokenizer = AutoTokenizer.from_pretrained("xlm-roberta-base")
model = AutoModelForSequenceClassification.from_pretrained(
            "xlm-roberta-base", num_labels=3)
training_args = TrainingArguments(
    output_dir=f"{BASE_DIR}/sentiment_model_checkpoints",
    eval_strategy="epoch", save_strategy="epoch",
    learning_rate=2e-5, per_device_train_batch_size=32,
    num_train_epochs=10, weight_decay=0.01, warmup_ratio=0.1,
    load_best_model_at_end=True, metric_for_best_model="f1",
    fp16=True, save_total_limit=1, save_only_model=True)
```

Three practical features make training on free cloud GPUs reliable:

1. Automatic batch-size selection. The notebook reads the GPU's memory and chooses the batch size and gradient accumulation (Section 3.3.1), so the same notebook runs on a T4, on a smaller GPU or on a CPU.
2. Resuming from checkpoints. Before training, the notebook looks for checkpoint folders on Google Drive, deletes any that are incomplete (missing trainer_state.json), and resumes from the latest complete checkpoint. A Colab disconnection therefore costs at most one epoch.
3. Disk-space control. Only the best checkpoint is kept (save_total_limit = 1) and optimiser states are not saved (save_only_model = True), which keeps the checkpoint folder within the free Google Drive quota.

After training, the reporting helper reads the Trainer's log history and produces the per-epoch table and the training curves (Figure 6.1), and saves them as epoch_metrics.csv and training_curves.png. The evaluation cell produces the classification report and the confusion matrix (Figure 6.2).

5.2.3 Sentiment prediction

Listing 5.3 shows the prediction function used by the notebook's interactive cell and by the service. The text is tokenised in the same way as during training, the model's logits are converted into probabilities with the softmax function, and the class with the highest probability is returned together with its probability.

Listing 5.3: Sentiment prediction function

```
def predict_sentiment(text, model, tokenizer, device):
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
                       max_length=128, padding=True).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
        pred_idx = torch.argmax(probs, dim=-1).item()
        confidence = probs[0][pred_idx].item()
    label_map = {0: "Negative", 1: "Neutral", 2: "Positive"}
    return label_map[pred_idx], confidence
```

5.2.4 Trust-score calculation

The scoring logic in the FastAPI service implements the formulas of Section 4.4. Listing 5.4 expresses this logic in Python.

Listing 5.4: Trust-score calculation (logic of the scoring service)

```
PRIOR_MEAN, PRIOR_WEIGHT = 6.0, 5
W_STAR, W_SENT = 0.60, 0.40
POLARITY = {"Negative": -1.0, "Neutral": 0.0, "Positive": 1.0}
def review_sentiment(label, confidence):
    return POLARITY[label] * confidence          # s_i in [-1, +1]
def trust_score(stars, sentiments):
    # stars: all star ratings of one worker in one category
    # sentiments: s_i of the reviews in that category that have text
    n = len(stars)
    if n == 0:
        return PRIOR_MEAN                        # new worker
    star10 = sum(stars) / n / 5 * 10
    if sentiments:
        s_bar = sum(sentiments) / len(sentiments)
        sent10 = 5.0 * (s_bar + 1.0)
        raw = W_STAR * star10 + W_SENT * sent10
    else:
        raw = star10                             # stars-only feedback
    c = n / (n + PRIOR_WEIGHT)                   # credibility
    return c * raw + (1 - c) * PRIOR_MEAN
```

5.3 Integration with the other components

Component 3 is integrated with the rest of the system at three points.

1. Node.js backend and mobile application. When a customer opens a completed job in the mobile application, the review screen collects the star rating and the optional text. The backend checks that the job is completed before it accepts the review, and then passes the review to the Component 3 FastAPI service. This enforces the verified-completion rule at the backend, before any analysis takes place.

2. MongoDB Atlas. The review, its sentiment value and the recomputed category score are stored in the shared MongoDB Atlas database, keyed by worker and service category, so that the scores persist and can be inspected or recomputed.
3. Component 2. The Hybrid Worker Recommendation engine needs a "Ratings score" for each candidate worker in the requested category. It obtains the trust score from Component 3 through the FastAPI API and uses it, together with the historical suitability score and the live GPS distance, in its ranking formula. Because Component 3 already isolates scores by category and adjusts them for the number of reviews, Component 2 can use the value directly, without its own handling of new workers or category mixing.

Component 1 does not call Component 3 directly, but the two are linked through the service category: the category that Component 1 predicts for a request becomes the category of the job, and therefore the category under which the job's review is scored.

5.4 Verification and software testing

Testing of Component 3 was carried out at four levels: evaluation of the model on the held-out test set, behavioural smoke tests, verification of the scoring formula, and integration testing with the application. Table 5.2 summarises the test plan and its status.

Table 5.2: Test plan and status

Level	What was tested	Method	Status
Model evaluation	Accuracy, F1, per-class metrics, confusion matrix on 16,853 test reviews	Automated, in the training notebook	Done (Section 6.1)
Behavioural smoke tests	Nine hand-written sentences: positive, negative and neutral in English, Sinhala and Singlish	Automated, at the end of training	Done (Table 6.4)
Interactive tests	Further Singlish sentences typed by hand	Manual, in the notebook	Done (Section 6.1.4)
Scoring formula	New worker, few and many reviews, stars-only reviews, star-text conflict, poor and excellent workers	Hand calculation checked against the Python logic	Done (Table 6.5)
Integration	Review submitted in the app; sentiment classified; score saved and available to Component 2	Manual end-to-end test with the app on localhost and the deployed model	Done, informally

Level	What was tested	Method	Status
User acceptance	Customers and workers using the application	Survey or field trial	Not done (Section 6.3)
Performance	Response time and throughput of the service	Load testing	Not done

The scoring formula verification cases are listed in Table 5.3. The expected values were calculated by hand from the formulas in Section 4.4 and match the output of the scoring logic in Listing 5.4.

Table 5.3: Scoring formula verification cases

Case	Input	Expected behaviour	Expected final score	Result
TC1	No reviews	Neutral starting score	6.00	Pass
TC2	1 review, 5 stars, Positive at 0.99 confidence	Strong review, low credibility; score rises only slightly	6.66	Pass
TC3	10 reviews, mean 4.2 stars, $\bar{S}$ = +0.40	Worked example of Section 4.4.5	7.23	Pass
TC4	10 reviews, mean 4.2 stars, no text	Stars only; Raw = Star10 = 8.40	7.60	Pass
TC5	10 reviews, mean 4.9 stars, $\bar{S}$ = -0.60	Complaints in the text lower a high star score	6.45	Pass
TC6	20 reviews, mean 2.1 stars, $\bar{S}$ = -0.70	Poor worker scores well below the prior	3.70	Pass
TC7	50 reviews, mean 4.8 stars, $\bar{S}$ = +0.85	Excellent, well-established worker	9.15	Pass
TC8	Neutral review text	$s_i = 0$ regardless of confidence	$s_i = 0$	Pass

The integration test was carried out manually by the author with the mobile application and the Node.js backend running on localhost and connected to the deployed model service. No automated unit tests or API tests were written for the service; adding them is part of the future work.

5.5 Deployment and operation

The sentiment model and the scoring logic are deployed together as a FastAPI service on a cloud server. When the service starts, it loads the saved model and tokenizer from the final_model folder once and keeps them in memory, so each request only runs a forward pass. The prediction code selects a GPU when one is available and otherwise runs on the CPU; a single forward pass of xlm-roberta-base on a short review is light

enough for CPU serving at the low request rates expected from review submissions. The rest of the application (the mobile application and the Node.js backend) has been run and tested on localhost, connected to the deployed model service and to MongoDB Atlas.

For operation in production, the following practices are planned:

- Version the model files, so that a retrained model can be deployed and rolled back without changing the API.
- Recompute all category scores when the scoring constants or the model change, using the stored star ratings and sentiment values.
- Log predictions with low confidence for later human review and relabelling.
- Monitor the distribution of predicted labels over time to detect drift between the training data and real reviews.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Training progress

Table 6.1 shows the training loss, validation loss, accuracy and weighted F1 at the end of each of the ten epochs, and Figure 6.1 plots them. The validation metrics were computed on the 16,853-review test set.

Table 6.1: Per-epoch training results

Epoch	Training loss	Validation loss	Accuracy	Weighted F1
1	0.7829	0.5996	0.7629	0.7630
2	0.4739	0.3788	0.8606	0.8593
3	0.2627	0.2631	0.9082	0.9083
4	0.1675	0.2480	0.9289	0.9289
5	0.1192	0.2306	0.9372	0.9372
6	0.0936	0.2405	0.9448	0.9447
7	0.0687	0.2973	0.9468	0.9468
8	0.0505	0.2950	0.9489	0.9489
9	0.0382	0.2843	0.9519	0.9519
10	0.0257	0.2957	0.9541	0.9541

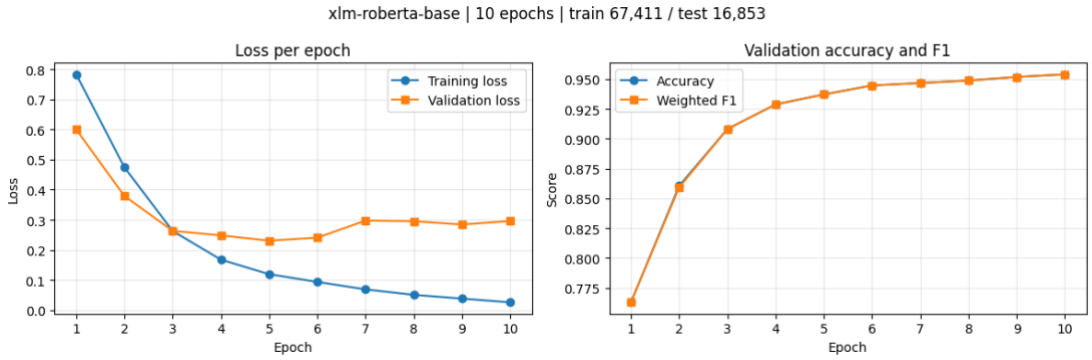


Figure 6.1: Training and validation loss, accuracy and weighted F1 per epoch

Accuracy rose quickly in the first three epochs, from 76.3% to 90.8%, and then more slowly, reaching 95.4% at epoch 10. Weighted F1 followed accuracy almost exactly, which is expected when the classes are fairly balanced. The training loss fell steadily from 0.78 to 0.03. The validation loss fell to its lowest value of 0.2306 at epoch 5, then rose to between 0.28 and 0.30 in epochs 7 to 10, while the training loss kept falling. This pattern shows that the model began to overfit after epoch 5: it kept getting more of the test reviews right, but it became more confident in its wrong answers, which

raises the cross-entropy loss. Because the best checkpoint was selected by weighted F1, the epoch-10 model was kept as the final model.

6.1.2 Overall and per-class performance

The final model reached an accuracy of 95.41%, a weighted F1 of 0.9541 and a macro F1 of 0.9539 on the test set, with a test loss of 0.2957. Table 6.2 gives the per-class results.

Table 6.2: Per-class results of the final model on the test set

Class	Precision	Recall	F1 score	Support
Negative	0.9580	0.9585	0.9582	5,971
Neutral	0.9466	0.9556	0.9511	4,840
Positive	0.9563	0.9485	0.9524	6,042
Macro average	0.9536	0.9542	0.9539	16,853
Weighted average	0.9541	0.9541	0.9541	16,853

All three classes have F1 scores between 0.95 and 0.96. Negative is the best-recognised class (F1 0.958). Neutral has the lowest precision (0.947), meaning that some reviews predicted as Neutral are actually positive or negative. Positive has the lowest recall (0.949), meaning that some positive reviews are missed.

6.1.3 Confusion matrix

Figure 6.2 and Table 6.3 show the confusion matrix. Of the 16,853 test reviews, 16,079 were classified correctly and 774 incorrectly.

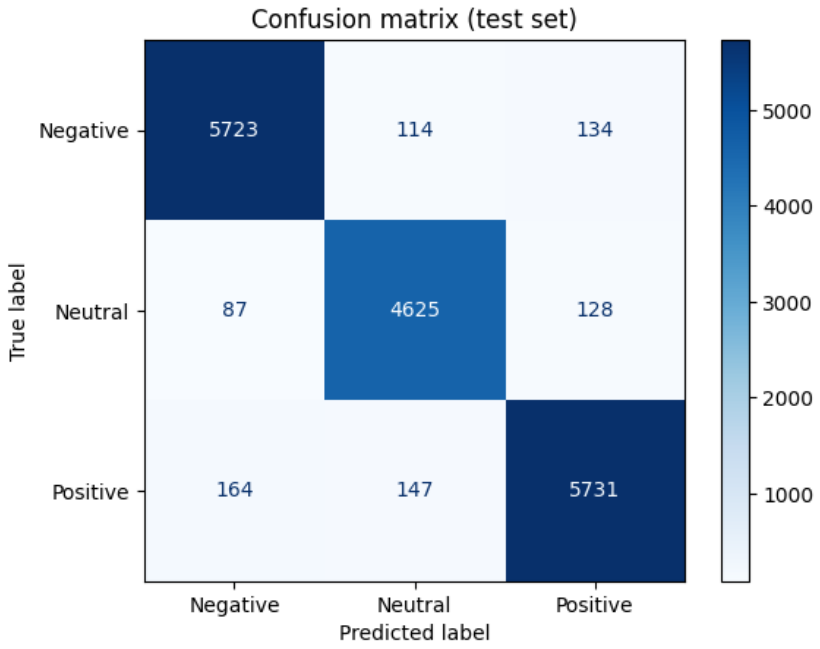


Figure 6.2: Confusion matrix of the final model on the test set

Table 6.3: Confusion matrix (rows are true labels, columns are predicted labels)

True label	Predicted Negative	Predicted Neutral	Predicted Positive	Total
Negative	5,723	114	134	5,971
Neutral	87	4,625	128	4,840
Positive	164	147	5,731	6,042

The largest single error is Positive reviews predicted as Negative (164 cases, 2.7% of positive reviews), followed by Positive predicted as Neutral (147) and Negative predicted as Positive (134). Errors in which a review is assigned the opposite polarity (Positive to Negative or Negative to Positive, 298 cases in total) matter most for the trust score, because they move a review's sentiment value from about +1 to about -1 or the reverse. They make up 1.8% of the test set.

6.1.4 Behavioural smoke tests

Table 6.4 shows the predictions of the final model for the nine smoke-test sentences and for two Singlish sentences entered in the interactive test.

Table 6.4: Smoke-test and interactive-test predictions

Language	Sentence	Expected	Predicted (confidence)	Correct
English	This game is absolutely wonderful and I enjoy playing it!	Positive	Positive (99.99%)	Yes

Language	Sentence	Expected	Predicted (confidence)	Correct
English	I hate the new update, it completely broke my gameplay.	Negative	Negative (99.99%)	Yes
English	The developers released a patch today.	Neutral	Negative (86.03%)	No
Sinhala	මේ ක්‍රීඩාව ඇත්තටම නියමයි, මම ගොඩක් ආසාවෙන් සෙල්ලම් කරනවා!	Positive	Positive (99.99%)	Yes
Sinhala	අළුත් අප්ඩේට් එක මල ජරාවක්, ක්‍රීඩා කරන්න බෑ.	Negative	Negative (99.99%)	Yes
Sinhala	අද අලුත් සංස්කරණයක් නිකුත් වුණා.	Neutral	Positive (99.69%)	No
Singlish	meka nam supiri game ekak bn, patta!	Positive	Positive (99.99%)	Yes
Singlish	aluth update eka hena upset, epa wenawa sllm krnna.	Negative	Negative (99.99%)	Yes
Singlish	Ada aluth patch ekak thiyanawa.	Neutral	Neutral (99.75%)	Yes
Singlish, interactive	wadak na ade mu	Negative	Negative (99.99%)	Yes
Singlish, interactive	meyanm hari hoday	Positive	Positive (99.99%)	Yes

The model classified all positive and negative sentences correctly in all three languages, nearly always with more than 99.9% confidence, including the informal Singlish sentences "wadak na ade mu" and "meyanm hari hoday", whose spellings do not follow any standard. It failed on two of the three neutral sentences. The English sentence about a patch release was labelled Negative and the Sinhala sentence about a new version release was labelled Positive with 99.69% confidence, while the equivalent Singlish sentence was correctly labelled Neutral. Overall, 9 of the 11 sentences were classified correctly. The two errors and the very high confidence values agree with the overfitting seen in the validation loss: the model is often more certain than it should be, especially for neutral, factual statements.

6.1.5 Answer to RQ1

A single fine-tuned XLM-R model classified the sentiment of the mixed English, Sinhala and Singlish test set with 95.41% accuracy and a weighted F1 of 0.954, above the 85% target set in Section 1.5. The behavioural tests confirm that it handles positive and negative text in all three forms, including informal Singlish spelling, but that it is

less reliable on neutral statements. Section 7.5 explains why the test-set figure is likely to be optimistic for real service reviews.

6.2 Product and technical results

The scoring model was evaluated with eight scenarios that cover the situations described in the requirements. Table 6.5 lists the inputs and the intermediate and final values, and Figure 6.3 compares the raw and final scores.

Table 6.5: Trust-score scenarios

Scenario	N	Mean stars	S̄	Star10	Sent10	Raw	C	Final
S1 New worker	0	-	-	-	-	-	0.000	6.00
S2 One excellent review	1	5.0	+0.99	10.00	9.95	9.98	0.167	6.66
S3 Three excellent reviews	3	5.0	+0.95	10.00	9.75	9.90	0.375	7.46
S4 Worked example	10	4.2	+0.40	8.40	7.00	7.84	0.667	7.23
S5 Stars only, no text	10	4.2	none	8.40	-	8.40	0.667	7.60
S6 High stars, negative text	10	4.9	-0.60	9.80	2.00	6.68	0.667	6.45
S7 Poor worker	20	2.1	-0.70	4.20	1.50	3.12	0.800	3.70
S8 Excellent, established worker	50	4.8	+0.85	9.60	9.25	9.46	0.909	9.15

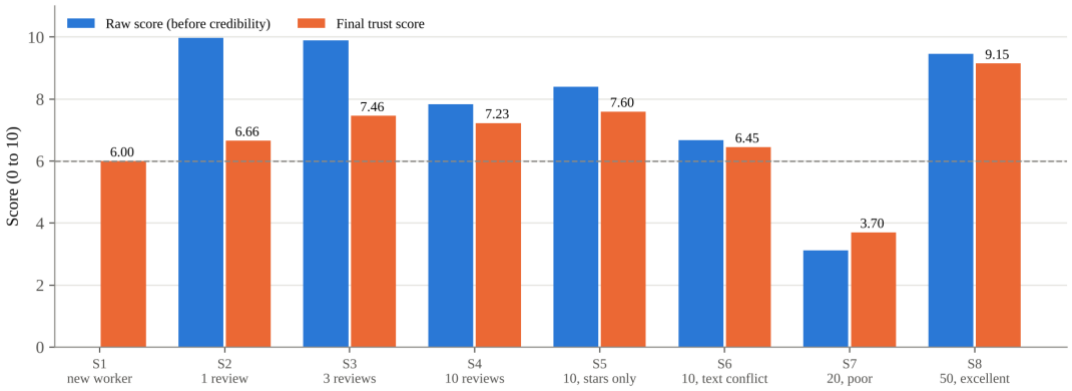


Figure 6.3: Raw and final trust scores for the eight scenarios

The scenarios show the intended behaviour:

- Starting point (S1). A worker without reviews in a category starts at 6.0, an acceptable but unproven score.
- Protection against a single review (S2, S3). One perfect review raises a new worker only from 6.00 to 6.66, and three raise the worker to 7.46. A lucky first review

cannot place a new worker above an established worker with a strong record (S8, 9.15), and an unfair first review cannot push a new worker to the bottom.

- Reward for an established record (S8). With 50 reviews the credibility is 0.91 and the final score of 9.15 is close to the raw score of 9.46.
- Use of the text (S4, S6). In S4, the mildly positive text lowers the raw score from 8.40 (stars alone) to 7.84. In S6, a worker with near-perfect stars but complaining reviews gets a raw score of 6.68 instead of 9.80; the disagreement between stars and text is reflected in the score, which a star-only system would not show.
- Stars-only feedback (S5). When no review has text, the score is computed from the stars alone, as required by FR6. The same star record with mildly positive text (S4) scores slightly lower, 7.23 compared with 7.60, because the text adds evidence that the experience was less than excellent.
- Poor performance (S7). A worker with low stars and negative text ends well below the prior, at 3.70, and further below it as reviews accumulate.

This answers RQ2 and RQ3: the fixed-weight fusion carries disagreement between stars and text into the score, and the Bayesian credibility adjustment keeps scores based on few reviews close to the prior, while letting workers with many reviews reach scores close to their own record.

6.3 User or stakeholder validation

No validation with real customers or workers has been carried out yet. The application has not been released to users, so there are no real reviews to test the model on and no user feedback on whether the trust scores match customers' impressions of workers. A planned validation would ask a group of customers to rate sample worker profiles and compare their judgements with the trust scores, and would collect real Singlish service reviews to build a hand-labelled test set (Section 9.4).

6.4 Integration results

The component was integrated with the mobile application, the Node.js backend, MongoDB Atlas and Component 2. In manual end-to-end testing with the application on localhost and the deployed model service, a review submitted for a completed job was classified, stored with its sentiment value, and the worker's category trust score

was updated and made available to Component 2 through the FastAPI API. Response times and behaviour under load were not measured.

6.5 Summary of findings

The main findings are:

1. A fine-tuned xlm-roberta-base model reached 95.41% accuracy, 0.954 weighted F1 and 0.954 macro F1 on 16,853 held-out reviews in English, Sinhala script and Singlish, with per-class F1 between 0.951 and 0.958.
2. Validation loss was lowest at epoch 5 and rose afterwards, showing overfitting and overconfidence in later epochs, although accuracy continued to improve slightly.
3. In behavioural tests, all positive and negative sentences were classified correctly in all three languages, including informal Singlish; two of three neutral sentences were misclassified with high confidence.
4. The scoring model behaves as designed: new workers start at 6.0, single reviews have a limited effect, established workers reach scores close to their record, stars-only feedback is handled, and disagreement between stars and text lowers the score.
5. The component works end to end with the application and supplies category-specific scores to Component 2, but it has not yet been validated with real users or real reviews.

7 DISCUSSION

7.1 Interpretation of findings

The results show that one multilingual transformer can learn three-class sentiment for English, Sinhala-script and Singlish text from a dataset that was largely created by augmentation. Two properties of XLM-R explain this. Its pre-training on 100 languages, including Sinhala, gives it a representation of Sinhala script from the start [2]. Its SentencePiece tokenizer breaks any romanised word into sub-word pieces [32], so Singlish words such as "supiri", "patta" or "hoday" are never "unknown", and the model can learn which pieces carry sentiment. The correct, highly confident predictions for "wadak na ade mu" and "meyanm hari hoday", whose spellings follow no standard and appear in no template, suggest that the model learned more than a list of slang words.

The weaker handling of neutral sentences has several likely causes. Neutral is the smallest of the three classes in the training data (19,358 of 67,411 training rows), and neutral tweets in the base dataset often contain opinions about an entity, so the boundary between Neutral and the other classes in the data is not sharp. The overfitting seen after epoch 5 makes the problem worse: an overconfident model assigns near-certain probabilities even when the text has no clear sentiment, which matches the general finding that modern neural networks are poorly calibrated [36]. In the scoring model, this matters less than it might seem, because a wrong label with high confidence on a factual review moves one review's sentiment value, and that value is averaged with the other reviews, weighted at 40%, and then shrunk by the credibility factor.

The very high test accuracy has to be read with care. The test set comes from the same augmented dataset as the training set. When an English tweet, its Sinhala translation and its Singlish transliteration are split at random, some versions can end up in the training set and others in the test set, so the model may be tested on near-copies of training examples. The template Singlish sentences are also built from a small vocabulary, so many test sentences closely resemble training sentences. Both effects are forms of data leakage that inflate test scores [37]. The test set was also used to choose the best epoch. The 95.41% accuracy therefore measures how well the model

fits this dataset, and the accuracy on real Sri Lankan service reviews is expected to be lower.

For the scoring model, the scenarios confirm that the design meets its goals. The fusion weights let text reduce an inflated star rating (scenario S6), which addresses the documented disagreement between ratings and review text [3]. The Bayesian adjustment removes the instability of small-sample averages that platforms such as Uber acknowledge [6], without hiding new workers or requiring a minimum number of reviews.

7.2 Comparison with literature baselines and existing solutions

Table 7.1 places the sentiment results next to related work. The comparison is only indicative, because the datasets, languages and numbers of classes differ.

Table 7.1: Comparison with related sentiment studies

Study	Language and data	Model	Classes	Reported result
Senevirathne et al. [13]	Sinhala news comments	RNN, LSTM, capsule networks	4	About 63% weighted accuracy
Rizvi et al. [16]	Sinhala and code-mixed text	Fine-tuned XLM-R	3	88.4% (Sinhala and code-mixed); 92.3% accuracy with BERT for English
Barbieri et al. [12]	Multilingual tweets, 8 languages	XLM-T (XLM-R further trained on tweets)	3	Strong multilingual baseline for tweet sentiment
This work	English, Sinhala script, transliterated and template Singlish (augmented tweets)	Fine-tuned xlm-roberta-base	3	95.41% accuracy, 0.954 weighted F1

The result of this work is higher than the published results for Sinhala and code-mixed sentiment. Part of the difference comes from the model: transformers clearly outperform the earlier recurrent and capsule models used on Sinhala news comments. A larger part probably comes from the data: the augmented test set contains text generated in the same way as the training set, while the other studies test on text written by real users. The results are therefore not directly comparable. They show that the approach works, not that this model is better than the others on real data.

Compared with existing rating systems (Table 2.1), the component differs in three ways. It uses the review text, in three language forms, as part of the score. It adjusts the score for the number of reviews with a transparent Bayesian average instead of showing a raw average. And it keeps a separate score for each service category. Like Uber and Airbnb, it relies on verified completion to keep fake reviews out.

7.3 Research contribution

The research contributions of this component are:

1. A multilingual three-class sentiment dataset of 101,217 rows that adds Sinhala-script, transliterated Singlish and template Singlish text to a public English dataset, together with a reusable rule-based Sinhala-to-Singlish transliteration function and template generator.
2. Evidence that a single fine-tuned xlm-roberta-base model can classify English, Sinhala and Singlish sentiment with high accuracy on such data (95.41% accuracy, 0.954 weighted F1), and a documented account of where it fails (neutral statements and overconfidence).
3. A transparent trust-scoring model that combines confidence-weighted text sentiment with star ratings, handles stars-only feedback, applies a Bayesian credibility adjustment and isolates scores by service category, together with worked scenarios that demonstrate its behaviour.

7.4 Product contribution

For the marketplace product, the component provides:

- A working FastAPI service that classifies review text and maintains category-specific trust scores, integrated with the Node.js backend and MongoDB Atlas.
- The "Ratings score" input that Component 2 needs, already normalised to 0 to 10, adjusted for new workers and separated by category, so that the recommendation engine can use it directly.
- A fairer experience for workers: new workers start from a neutral score and are protected from a single bad review, and a poor job in one trade does not affect a worker's reputation in another.

- A more informative experience for customers: worker scores reflect what previous customers wrote as well as the stars they selected, in the languages customers actually use.

7.5 Limitations and threats to validity

The main limitations and threats to validity are:

- Domain mismatch. The model was trained on tweets about brands and video games, not on reviews of household services. Words that signal sentiment about a plumber ("late", "leaking again", "cleaned up afterwards") may not be well represented.
- Synthetic Sinhala and Singlish data. The Sinhala text is unverified machine translation, and the transliterated Singlish follows formal written Sinhala (for example "meya ithaa hondayi") rather than the casual way people type Singlish ("meka hari hondai"). The template sentences use a small vocabulary. The model may therefore perform worse on real Singlish reviews than on the test set.
- Data leakage and model selection on the test set. The random, row-level split can place versions of the same tweet on both sides of the split, and the test set was also used to choose the best epoch. Both make the reported accuracy optimistic [37]. A split grouped by the original tweet identifier, with a separate validation set, would remove these effects.
- Unknown language composition. The number of rows of each language type was not recorded, so results cannot be broken down by language, and it is not known how much of the test accuracy comes from English rows.
- Overconfidence and neutral errors. Validation loss rose after epoch 5, and neutral statements were misclassified with high confidence in the smoke tests. Because the sentiment value is weighted by confidence, overconfident errors have full weight.
- Untuned scoring constants. The 60/40 weights, the prior mean of 6.0 and the prior weight of 5 were chosen by judgement, not estimated from data.
- Limited testing. There are no automated unit or API tests, no load or response-time tests, and no validation with real users or real reviews. The rest of the application has been tested only on localhost.

- Scope. Sarcasm, mixed reviews that praise one aspect and criticise another, Tamil and Tanglish reviews, and offensive content are not handled.

7.6 Implications

For the marketplace, the results imply that customer reviews written in Sinhala and Singlish can be used automatically, rather than ignored, and that a transparent, text-aware and sample-size-aware score is feasible with modest resources: the model trains in 80 minutes on a free cloud GPU and runs on an ordinary server. For workers, the design implies a fairer path into the market, because a new worker is neither hidden nor exposed to the full effect of a single review.

For research on Sinhala NLP, the work shows that augmentation by translation, transliteration and templates can bootstrap a multilingual classifier when no labelled Singlish data exists. It also shows that high accuracy on augmented data needs to be confirmed on real user text, with leakage-free splits, before it is reported as real-world performance. The most valuable next step is therefore not a larger model but a small, hand-labelled set of real Singlish service reviews.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Component 3 is not sold on its own in the first instance; it is part of the group's household-service marketplace, and its value is that it makes the marketplace's ratings trustworthy. It serves three groups of users.

- Customers want to know whether a worker will do a good job. The component gives them a single score that reflects both the stars and the written comments of previous customers, in English, Sinhala or Singlish, and that cannot be inflated by fake reviews or by a single lucky review.
- Workers want a fair chance to be chosen. The component gives every new worker a neutral starting score, limits the effect of any single review, and keeps their reputation in each trade separate, so that a skilled plumber is not penalised for one weak electrical job.
- The marketplace operator wants customers to return and good workers to stay. Reliable scores lead to better matches, fewer disputes and more repeat business, and they give the recommendation engine (Component 2) a dependable input.

The value proposition can be summarised as: "Trust scores that read what customers actually write, in the languages Sri Lankans actually use, and that are fair to new workers."

8.2 Market and competitive position

At the start of 2025 Sri Lanka had about 12.4 million internet users (53.6% of the population) and about 29.3 million mobile connections [1]. Household maintenance services are still found mainly through personal contacts and classified-advertisement websites, where listings rarely carry verified ratings. Ride-hailing and delivery apps have accustomed Sri Lankan users to rating a service after it is completed, which makes a rated service marketplace familiar.

Table 8.1 compares the trust features of the proposed marketplace with the main alternatives.

Table 8.1: Competitive comparison of trust features

Feature	Word of mouth	Classified-advertisement sites	Global service marketplaces	Proposed marketplace (with Component 3)
Verified reviews only	Not applicable	No	Usually yes	Yes
Score uses review text	No	No	No (text shown separately)	Yes
Sinhala and Singlish support	Yes (spoken)	Partial (listing text)	No	Yes
Adjustment for few reviews	No	No	Usually not shown	Yes (Bayesian credibility)
Separate score for each trade	Informal	No	Partly	Yes

The competitive advantage of the component is its combination of local-language understanding and a transparent, fair scoring method. A global platform entering Sri Lanka would need to build Sinhala and Singlish support; a local classified site would need to add verified jobs, ratings and text analysis. The trained model, the augmentation method and the Singlish data give the project a head start.

8.3 Business or sustainability model

Within the marketplace, the component supports a revenue model such as a small commission on each completed job or optional paid subscriptions for workers. Whichever model the marketplace adopts, it depends on trust: if customers stop trusting ratings, they return to word of mouth and the marketplace loses its income. Accurate and fair trust scores therefore protect the marketplace's revenue.

The component also has stand-alone potential as a "review intelligence" API for other Sri Lankan businesses that collect customer feedback in Sinhala and Singlish, such as ride-hailing, food delivery, e-commerce and hotel booking services. Possible pricing models are:

- A free tier with a monthly limit of analysed reviews, for small businesses and developers.
- Paid tiers priced per thousand analysed reviews, including category-level trust scores and dashboards.

- Custom fine-tuning of the model on a client's own labelled reviews for a one-time fee.

The running costs are low. The base model runs on an ordinary cloud server without a GPU at the expected request rates, and retraining takes about 80 minutes on a single T4 GPU, which is available free or at low cost from cloud notebook services. This keeps the service financially sustainable even with modest revenue.

8.4 Scalability intellectual property and compliance

Scalability. The FastAPI service holds the model in memory and does not keep any per-request state, so more instances can be started behind a load balancer as the number of reviews grows. Scores are recomputed only when a new review arrives for a worker and category, and the intermediate values are stored, so recomputation is cheap. MongoDB Atlas is a managed database that can be scaled without changing the application [59]. If request volume grows substantially, the model can be made faster with techniques such as quantisation or distillation into a smaller model.

Intellectual property. The pre-trained xlm-roberta-base model is released under the MIT licence, which permits commercial use. The fine-tuned model weights, the augmentation and transliteration code, the scoring model and the service code were created by the project and can be protected as trade secrets or released under a licence of the group's choice. Before commercial use of the training data, the licence of the Kaggle base dataset must be checked, and Google Translate's terms of service must be respected; replacing the translated data with the project's own real, consented review data would remove both dependencies.

Compliance. Reviews are personal data under the Personal Data Protection Act, No. 9 of 2022 [54]. Before launch, the marketplace will need a privacy notice that explains how reviews are used to compute trust scores, a lawful basis for processing, a data retention period, a process for workers to see and question their scores, and security controls for the database. The design already supports data minimisation, because only aggregated scores are shared with other components.

8.5 Contribution to the Sustainable Development Goals

The component contributes to several of the United Nations Sustainable Development Goals [60]:

- SDG 8, Decent Work and Economic Growth. Target 8.3 calls for policies that support decent job creation and encourage the formalisation and growth of micro and small enterprises. A fair, verified reputation gives informal workers a portable record of their quality, which helps them win more and better-paid work.
- SDG 10, Reduced Inequalities. New workers, and workers who lack personal networks, are often excluded from word-of-mouth markets. The neutral starting score and the protection against single reviews give them a fairer chance to compete with established workers. Support for Sinhala and Singlish means that customers who do not write in English are also heard.
- SDG 9, Industry, Innovation and Infrastructure. The component applies modern multilingual AI to a local, low-resource language setting and shows that it can be done with modest computing resources.
- SDG 12, Responsible Consumption and Production. Better matching of customers and workers reduces failed or repeated repairs, saving materials and travel.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented Component 3 of the R26-IT-090 smart service marketplace: Automated Review Credibility and Sentiment Analysis. The component addresses three weaknesses of star-only ratings in a Sri Lankan household-service marketplace: stars that disagree with what customers write, averages that rest on too few reviews, and written feedback in English, Sinhala and Singlish that English-only tools cannot read.

A three-class sentiment dataset of 101,217 rows was built by extending a public English Twitter dataset with Sinhala machine translations, rule-based Singlish transliterations and template Singlish sentences. A fine-tuned xlm-roberta-base model reached 95.41% accuracy and a weighted F1 of 0.954 on 16,853 held-out examples, and it classified positive and negative sentences correctly in all three languages, including informal Singlish. A trust-scoring model fuses star ratings (60%) and confidence-weighted text sentiment (40%) into a 0 to 10 score, falls back to stars alone when no review has text, adjusts the score for the number of reviews with a Bayesian credibility factor $C = N/(N + 5)$ and a prior of 6.0, and keeps a separate score for each service category. Reviews are accepted only for completed jobs. The model and the scoring logic run as a FastAPI service, store results in MongoDB Atlas and supply the trust score to the recommendation engine through an API.

The work also identified clear limits. The test accuracy is likely to be optimistic because the data is augmented, comes from tweets rather than service reviews, and may leak across the train-test split. The model is overconfident and weaker on neutral statements. The scoring constants were set by judgement, and the component has not yet been tested with real users.

9.2 Achievement of objectives

Table 9.1 summarises how far each objective of Section 1.5 was achieved.

Table 9.1: Achievement of objectives

Objective	Outcome	Status
1. Prepare a three-class English, Sinhala and Singlish sentiment dataset	101,217-row augmented dataset; 84,264 rows used after removing Irrelevant	Achieved
2. Fine-tune and evaluate XLM-R, target at least 85% accuracy	95.41% accuracy, 0.954 weighted F1, per-class F1 0.951 to 0.958; confusion matrix and smoke tests reported	Achieved, with the caveats of Section 7.5
3. Design a scoring model that fuses stars and sentiment, handles reviews without text and adjusts for the number of reviews	60/40 fusion, stars-only fallback, Bayesian credibility adjustment; verified with eight scenarios	Achieved
4. Keep scores separate for each category and accept reviews only for completed jobs	Category isolation and verified-completion rule implemented	Achieved
5. Implement as a FastAPI service with MongoDB Atlas and deliver the score to Component 2	Model and scoring deployed as a FastAPI service; integrated with the application and Component 2	Achieved; tested manually, not under load or with real users

9.3 Individual research and product contributions

The author's individual contributions are:

- The design of the component's role and its interface with Component 2.
- The multilingual dataset, including the Sinhala translation pipeline, the rule-based Sinhala-to-Singlish transliteration function and the template Singlish generator.
- The fine-tuning, evaluation and reporting of the XLM-R sentiment model, including the robust Colab training notebook with automatic batch sizing and checkpoint recovery.
- The trust-scoring model, combining confidence-weighted sentiment, star ratings, a stars-only fallback, Bayesian credibility and category isolation.
- The FastAPI service that hosts the model and the scoring logic, and its integration with the Node.js backend, MongoDB Atlas and Component 2.

9.4 Future directions

The following work would strengthen the component:

1. Real review data. Collect real customer reviews from the application, with consent, and label a test set of a few hundred Singlish, Sinhala and English service reviews by hand. Report accuracy on this set separately for each language form.
2. Leakage-free evaluation. Retrain with a split grouped by the original tweet identifier (for example with scikit-learn's GroupShuffleSplit), with separate validation and test sets, and record the number of rows of each language type.
3. Domain adaptation. Fine-tune further on real service reviews once they are available, and add service-specific vocabulary to the augmentation templates.
4. Calibration and early stopping. Stop training near the epoch with the lowest validation loss and calibrate the output probabilities, for example with temperature scaling [36], so that confidence values reflect real accuracy before they weight the sentiment values.
5. Tuning the scoring constants. Once real data is available, estimate the prior mean from the actual distribution of worker scores, and choose the weights and the prior strength by comparing trust scores with customer judgements or repeat bookings.
6. Richer analysis. Add aspect-based sentiment (punctuality, quality, price, cleanliness), sarcasm handling, and support for Tamil and Tanglish reviews.
7. Recency. Give recent reviews more weight than old ones, for example with exponential decay [20], so that scores reflect a worker's current performance.
8. Engineering. Add automated unit and API tests, measure response times under load, version the model files and monitor prediction drift in production.
9. User validation. Carry out a survey or field trial with customers and workers to test whether the trust scores match their judgements and are perceived as fair.

REFERENCES

[1] S. Kemp, "Digital 2025: Sri Lanka," DataReportal, Mar. 2025. [Online]. Available: https://datareportal.com/reports/digital-2025-sri-lanka

[2] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2020, pp. 8440–8451.

[3] M. Fazzolari, V. Cozza, M. Petrocchi, and A. Spognardi, "A study on text-score disagreement in online reviews," Cognitive Computation, vol. 9, no. 5, pp. 689–701, 2017, doi: 10.1007/s12559-017-9496-y.

[4] N. Hu, J. Zhang, and P. A. Pavlou, "Overcoming the J-shaped distribution of product reviews," Commun. ACM, vol. 52, no. 10, pp. 144–147, 2009.

[5] G. Zervas, D. Proserpio, and J. W. Byers, "A first look at online reputation on Airbnb, where every stay is above average," Marketing Letters, vol. 32, no. 1, pp. 1–16, 2021, doi: 10.1007/s11002-020-09546-4.

[6] Uber Technologies Inc., "How ratings work," Uber Driver Basics. [Online]. Available: https://www.uber.com/gr/en/drive/basics/how-ratings-work/ (accessed Oct. 8, 2026).

[7] N. de Silva, "Survey on publicly available Sinhala natural language processing tools and research," arXiv:1906.02358, 2019.

[8] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination," in Proc. 49th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2011, pp. 309–319.

[9] M. Luca, "Reviews, reputation, and revenue: The case of Yelp.com," Harvard Business School, Working Paper 12-016, 2016.

[10] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2002, pp. 79–86.

[11] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2013, pp. 1631–1642.

[12] F. Barbieri, L. Espinosa Anke, and J. Camacho-Collados, "XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond," in Proc. 13th Lang. Resources Eval. Conf. (LREC), 2022, pp. 258–266.

[13] L. Senevirathne, P. Demotte, B. Karunanayake, U. Munasinghe, and S. Ranathunga, "Sentiment analysis for Sinhala language using deep learning techniques," arXiv:2011.07280, 2020.

[14] H. Rathnayake, J. Sumanapala, R. Rukshani, and S. Ranathunga, "Adapter-based fine-tuning of pre-trained multilingual language models for code-mixed and code-switched text classification," Knowledge and Information Systems, vol. 64, no. 7, pp. 1937–1966, 2022.

[15] S. Chathuranga and S. Ranathunga, "Classification of code-mixed text using capsule networks," in Proc. Int. Conf. Recent Advances Natural Lang. Process. (RANLP), 2021, pp. 256–263.

[16] A. Rizvi, N. Thamindu, A. M. N. H. Adhikari, W. P. U. Senevirathna, D. Kasthurirathna, and L. Abeywardhana, "Enhancing multilingual sentiment analysis with explainability for Sinhala, English, and code-mixed content," arXiv:2504.13545, 2025.

[17] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, "Reputation systems," Commun. ACM, vol. 43, no. 12, pp. 45–48, 2000.

[18] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," Decision Support Systems, vol. 43, no. 2, pp. 618–644, 2007.

[19] E. Miller, "How not to sort by average rating," 2009. [Online]. Available: https://www.evanmiller.org/how-not-to-sort-by-average-rating.html

[20] E. Miller, "Bayesian average ratings," Nov. 2012. [Online]. Available: https://www.evanmiller.org/bayesian-average-ratings.html

[21] B. Liu, Sentiment Analysis and Opinion Mining. San Rafael, CA, USA: Morgan & Claypool, 2012.

[22] B. Pang and L. Lee, "Opinion mining and sentiment analysis," Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.

[23] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. 8th Int. AAAI Conf. Weblogs Social Media (ICWSM), 2014, pp. 216–225.

[24] Y. Kim, "Convolutional neural networks for sentence classification," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2014, pp. 1746–1751.

[25] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in Proc. 49th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2011, pp. 142–150.

[26] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015, pp. 649–657.

[27] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT), 2019, pp. 4171–4186.

[29] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[30] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune BERT for text classification?" in Proc. China Nat. Conf. Chinese Comput. Linguistics (CCL), LNCS vol. 11856, 2019, pp. 194–206.

[31] T. Pires, E. Schlinger, and D. Garrette, "How multilingual is multilingual BERT?" in Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2019, pp. 4996–5001.

[32] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in Proc. Conf. Empirical Methods Natural Lang. Process.: System Demonstrations (EMNLP), 2018, pp. 66–71.

[33] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learning Representations (ICLR), 2019.

[34] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods Natural Lang. Process.: System Demonstrations (EMNLP), 2020, pp. 38–45.

[35] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427–437, 2009.

[36] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 1321–1330.

[37] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Trans. Knowledge Discovery from Data, vol. 6, no. 4, pp. 1–21, 2012.

[38] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2020, pp. 4902–4912.

[39] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[40] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, Bayesian Data Analysis, 3rd ed. Boca Raton, FL, USA: CRC Press, 2013.

[41] I. U. Liyanage, "Sentiment analysis of Sinhala news comments," M.Sc. thesis, Dept. Comput. Sci. Eng., Univ. Moratuwa, Moratuwa, Sri Lanka, 2018.

[42] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, and M. Choudhury, "GLUECoS: An evaluation benchmark for code-switched NLP," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2020, pp. 3575–3585.

[43] P. Patwa et al., "SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets," in Proc. 14th Workshop Semantic Evaluation (SemEval), 2020, pp. 774–790.

[44] B. R. Chakravarthi, V. Muralidaran, R. Priyadharshini, and J. P. McCrae, "Corpus creation for sentiment analysis in code-mixed Tamil-English text," in Proc. 1st Joint

Workshop Spoken Lang. Technologies Under-resourced Languages (SLTU) and Collaboration Computing Under-Resourced Languages (CCURL), 2020, pp. 202–210.

[45] Y. De Mel, K. Wickramasinghe, N. de Silva, and S. Ranathunga, "Sinhala transliteration: A comparative analysis between rule-based and Seq2Seq approaches," in Proc. 1st Workshop Natural Lang. Process. Indo-Aryan and Dravidian Languages (IndoNLP), COLING 2025; arXiv:2501.00529.

[46] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP-IJCNLP), 2019, pp. 6382–6388.

[47] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2016, pp. 86–96.

[48] J. McAuley and J. Leskovec, "Hidden factors and hidden topics: Understanding rating dimensions with review text," in Proc. 7th ACM Conf. Recommender Systems (RecSys), 2013, pp. 165–172.

[49] S. M. Mudambi and D. Schuff, "What makes a helpful online review? A study of customer reviews on Amazon.com," MIS Quarterly, vol. 34, no. 1, pp. 185–200, 2010.

[50] N. Jindal and B. Liu, "Opinion spam and analysis," in Proc. Int. Conf. Web Search and Data Mining (WSDM), 2008, pp. 219–230.

[51] jp797498e, "Twitter Sentiment Analysis (entity-level sentiment dataset)," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis

[52] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[53] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019, pp. 8024–8035.

[54] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022. Colombo, Sri Lanka, 2022.

[55] M. Mitchell et al., "Model cards for model reporting," in Proc. Conf. Fairness, Accountability, and Transparency (FAT*), 2019, pp. 220–229.

[56] T. Gebru et al., "Datasheets for datasets," Commun. ACM, vol. 64, no. 12, pp. 86–92, 2021.

[57] Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[58] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 8, 2026).

[59] MongoDB Inc., "MongoDB Atlas documentation." [Online]. Available: https://www.mongodb.com/docs/atlas/ (accessed Oct. 8, 2026).

[60] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

GLOSSARY

Bayesian average: An average that combines an item's own ratings with a prior value, weighted as a fixed number of imaginary ratings, so that items with few ratings stay close to the prior.

Code-mixing: Using words and grammar from two or more languages within one sentence or conversation.

Credibility factor (C): In this thesis, N/(N + 5): the share of the final trust score that comes from the worker's own reviews rather than the prior.

Data augmentation: Creating additional training examples from existing ones, for example by translation or by filling templates.

Data leakage: Information from the test set reaching the model during training or model selection, which makes measured performance optimistic.

Fine-tuning: Continuing the training of a pre-trained model on a smaller, task-specific dataset.

Prior: The value a score starts from before any reviews are received; 6.0 out of 10 in this component.

Singlish: Sinhala written informally in the Latin alphabet, often mixed with English words.

Softmax: A function that turns a model's raw output scores into probabilities that sum to one.

SentencePiece: A tokenizer that splits text into sub-word units learned from data, so that any word can be represented.

Transliteration: Writing the words of one script in another script according to their sound, for example Sinhala script in Latin letters.

Trust score: The final 0 to 10 score of a worker in one service category produced by Component 3.

APPENDICES

Appendix A Extended results and research artefacts

A.1 Training run summary

Table A.1: Summary of the final training run

Item	Value
Platform	Google Colab, NVIDIA Tesla T4 (15,360 MiB), CUDA 13.0
Libraries	transformers 5.18.0; torch 2.11.0+cu130
Dataset file	twitter_training_fast_augmented.csv, 101,217 rows
Optional augmentation cell	Switched off (RUN_AUGMENTATION = False)
Rows after cleaning	84,264 (train 67,411; test 16,853)
Training classes (train)	Negative 23,883; Neutral 19,358; Positive 24,170
Batch configuration	Batch size 32, gradient accumulation 1, FP16 on
Steps	2,107 per epoch; 21,070 in total
Training time	80.0 minutes (1:19:55)
Best checkpoint	checkpoint-21070 (epoch 10), selected by weighted F1
Final test metrics	Loss 0.2957; accuracy 0.954073; weighted F1 0.954075
Saved outputs	final_model, epoch_metrics.csv, training_curves.png, classification_report.txt, confusion_matrix.png

A.2 Template Singlish generator settings

Table A.2: Settings of the template Singlish generator

Setting	Value
Slang words	10 positive, 10 negative, 6 neutral
Sentence templates	8 positive, 7 negative, 4 neutral
Typing variations	Repeated final letters ("i", "ii", "iii"), doubled vowels ("a" to "aa", "i" to "ii"), upper case
Emoji	Added with 70% probability, matching the polarity
Label	Copied from the source English row

A.3 Transliteration examples

Table A.3: Examples of the rule-based Sinhala-to-Singlish transliteration

Sinhala input	Singlish output
මෙය ඉතා හොඳයි	meya ithaa hondayi
මම ඔහුගේ සේවයට කැමති නැහැ	mama ohugee seewayata kaemathi naehae
ප්‍රශ්නයක් නැහැ	prashnayak naehae

Sinhala input	Singlish output
ශ්‍රී ලංකාව	shree lankaawa

Appendix B Academic integrity and AI use disclosure

The following tools were used in this work.

Table B.1: Use of AI and automated tools

Tool	Purpose	Sections or artefacts affected	Verification performed
Google Translate (through the deep-translator library)	Translating English training tweets into Sinhala as part of data augmentation	Training dataset (Section 3.2)	Translations were not checked systematically by a human; this limitation is reported in Section 7.5
Claude (Anthropic), a generative AI assistant	Helping to structure and draft the thesis text from the author's own code, notebook outputs and design decisions; producing the diagrams and charts from the author's results; formatting references	All chapters; Figures 1.1, 2.1, 3.1, 3.2, 4.1, 4.2 and 6.3	All numbers were checked against the training notebook outputs; all references were checked against their sources; the text was reviewed and edited by the author